EMBEDDED LINUX НА ПРАКТИКЕ · КНИГА 2

Разработка на основе SDK

Свои программы и пакеты, модули ядра, device tree, правки ядра и загрузчика — с примерами для ODROID-GO Advance, BeagleBone Black, Raspberry Pi 1 и Raspberry Pi 4/400

SDK `github.com/sobs1992/linux-sdk`
Linux v7.2.9 · U-Boot v2026.07 · Buildroot 2026.08
Примеры: `docs/dev-book/examples/`
Октябрь 2026

Содержание

Предисловие **4**
Примеры 4
Как читать 4

1. Карта разработки **6**
Где живут изменения 6
Что пересобирать 7
Как доставить на плату 7
Цикл разработки 8

2. Рабочее место **9**
Консоль UART 9
Сеть 9
Передача файлов по UART 10
Картридер 11
Тулчейн для своих программ 11

3. Быстрый цикл: загрузка по сети **12**
Сервер на ПК 12
PXE: тот же extlinux, но по сети 13
Ручная загрузка без PXE 14
Сделать сетевую загрузку постоянной 14
Цикл разработки с NFS 14
А если сети нет (OGA) 14

4. Свои программы **15**
Самая короткая дорога 15
Makefile-проект 15
CMake-проект 16
Библиотеки 16
Отладка 16
Что важно помнить 17

5. Программы для конкретных плат **18**
Светодиоды: без единой строки кода 18
GPIO: BeagleBone Black и Raspberry Pi 18
ODROID-GO Advance: кнопки и экран 20
ODROID-GO Advance: UART на разъёме 23
Звук 24

6. Программы в rootfs: пакеты Buildroot **26**
Включить готовый пакет 26
Свой пакет 27
Overlay: просто файлы 29
Быстрая проверка пакета отдельно 29

7. Модули ядра вне дерева **31**
Главное правило: тот же самый ядро 31
Общий Makefile 31
Пример 1: hello 32

Пример 2: символьное устройство 33
Пример 3: логгер кнопок (OGA и любая плата с клавиатурой)....34
Пример 4: драйвер по device tree....36
Модули в образе....38
Отладка модулей 38

8. Правки ядра и device tree 39
Порядок работы с исходниками ядра 39
Опции ядра 39
Свой драйвер в дереве ядра 39
Device tree: два способа....40
Отладка ядра....44

9. Правки в U-Boot.... 45
Какой U-Boot на самом деле работает 45
Как восстановиться, если U-Boot сломан....45
Окружение....45
Полезные команды....46
Пример: своя команда....46
Пример: меню с запасным ядром 48
Отладка самого U-Boot....48

10. Особенности плат: сводка для разработчика.... 49
Сравнение 49
ODROID-GO Advance....49
BeagleBone Black 50
Raspberry Pi 1 / Zero....50
Raspberry Pi 4 / 400 51

11. Практикум.... 52
Где держать свой код....52
Чек-лист перед коммитом 52
Задания 52

A. Шпаргалка разработчика 54
Программы....54
Пакеты....54
Модули....54
Ядро и device tree....54
U-Boot....54
Доставка....55

B. Указатель примеров 56
Что и как проверено....57

Предисловие

Первая книга рассказывала, как построен SDK: как собираются TF-A, U-Boot, ядро и rootfs и как они склеиваются в образ. Эта книга — о том, **что делать с SDK дальше**: как писать под платы свои программы, добавлять их в rootfs, писать модули ядра, описывать своё железо в device tree и вносить изменения в ядро и загрузчик.

Всё рассказывается для четырёх плат, которые поддерживает SDK:

Прошивка	Плата	Архитектура	Связь с ПК
`oga`	ODROID-GO Advance (Black Edition)	arm64, Cortex-A35	UART, USB-Ethernet адаптер, картридер
`bbb`	BeagleBone Black	arm, Cortex-A8	UART, Ethernet
`rpi1`	Raspberry Pi 1 / Zero	arm, ARM1176 (ARMv6)	UART, Ethernet (B/B+)
`rpi4`	Raspberry Pi 4 B / 400	arm64, Cortex-A72	UART, Ethernet, HDMI + клавиатура

Примеры

Все примеры лежат в репозитории SDK, в `docs/dev-book/examples/`, и подставляются в книгу прямо из файлов:

```
examples/
├── app/            программы: hello, cmake-demo, gpio-blink, oga-pad, serial-echo
├── package/        пакет Buildroot sdk-hello с init-скриптом
├── kmod/           модули ядра: hello, sdkchar, padlog, sdk-gpio-led (+ module.mk)
├── dts/            overlay device tree для BBB, RPi и OGA
└── uboot/          команда U-Boot sdkinfo
```

> **ЧТО ПРОВЕРЕНО**
>
> Все программы собраны тулчейнами Buildroot всех четырёх плат без предупреждений. Все модули собраны против ядер всех четырёх прошивок без предупреждений. Overlay скомпилированы и применены к настоящим dtb утилитой `fdtoverlay`. Команда U-Boot собрана для OGA и Raspberry Pi 1. Пакет Buildroot собран в отдельной тестовой сборке. **На железе** примеры не запускались — это ваша часть работы, и в книге отмечено, где что может пойти не так.

Как читать

- Главы 1–3 — рабочее место: где живут изменения, как подключаться к платам, как ускорить цикл «собрал — проверил» сетевой загрузкой.
- Главы 4–6 — пользовательские программы: кросс-компиляция, отладка, примеры для каждой платы, пакеты Buildroot.
- Главы 7–8 — ядро: модули вне дерева, правки в дереве, device tree и overlay.
- Глава 9 — U-Boot: свои команды, окружение, безопасная проверка.

- Глава 10 — особенности каждой платы одним списком.
- Глава 11 — практикум.

Предполагается, что первая книга прочитана или хотя бы знакомы главы про архитектуру SDK, Makefile и образ.

ГЛАВА 1

Карта разработки

Прежде чем писать код, полезно знать **где** будет жить каждое изменение, **что** придётся пересобрать и **как** доставить результат на плату. Неправильный ответ на любой из этих вопросов превращает пятиминутную правку в час возни.

Где живут изменения

Что меняете	Где хранить	Попадает на плату через
Своя программа, в разработке	где угодно, кросс-компиляция руками	`scp`, UART, картридер
Своя программа, готовая	`apps/<имя>/` + пакет в `board/common/buildroot/package/`	`make rootfs image`
Готовый пакет из Buildroot (strace, sdl2…)	`board/<плата>/buildroot/defconfig`	`make rootfs image`
Конфиги и скрипты в rootfs	`board/common/buildroot/rootfs-overlay/`	`make rootfs image`
Модуль ядра вне дерева	свой каталог (см. `examples/kmod/`)	`insmod` вручную или `make image`
Опции ядра	фрагмент `board/<плата>/linux/*.config`	`make linux image`
Драйвер, правка ядра	`src/linux`, ветка `oga`	`make linux image`
Узел device tree	`src/linux` (dts платы) или overlay	`make linux image` / `flash-boot`
Опции U-Boot	фрагмент `board/<плата>/u-boot/*.config`	`make uboot image` + `flash-boot`
Код U-Boot	`src/u-boot`, ветка `oga`	`make uboot image` + `flash-boot`
Аргументы ядра	`LINUX_CMDLINE` в конфиге прошивки	`make image` + `flash-boot`

Общее правило: **плата описывается в `board/` и `configs/`, код — в сабмодулях, результат — только в `output/`**. Ничего не правьте в `output/` руками, кроме экспериментов, которые не жалко потерять.

Что пересобирать

После правки в	Команда	Сколько времени
программе (руками)	`$CC ...`	секунды
пакете Buildroot	`make rootfs-make BR=<пакет>-rebuild && make rootfs image`	минута
overlay	`make rootfs image`	минута
модуле вне дерева	`make` в каталоге модуля	секунды
ядре	`make linux image`	от минуты (инкрементально) до 20 минут
U-Boot	`make uboot image`	1–2 минуты
конфиге прошивки	зависит от переменной; `make image` всегда	

ЛОВУШКА: `MAKE LINUX` СТИРАЕТ `LINUX-INSTALL/`

Цель `linux` очищает `output/<fw>/linux-install/` и ставит туда модули заново. Модули вне дерева, установленные туда раньше (`make install` в каталоге модуля), пропадут. Порядок всегда такой: `make linux` → установка своих модулей → `make image`. В главе 7 есть рецепт, как встроить это в SDK.

Как доставить на плату

Способ	Что меняет	OGA	BBB	RPi 1	RPi 4/400
`make flash`	всё	√	√	√	√
`make flash-boot`	загрузчик + ядро + dtb	√	√	√	√
Картридер: файлы на BOOT (FAT)	ядро, dtb, extlinux.conf, config.txt	√	√	√	√
Картридер: файлы в rootfs (ext4, `sudo mount`)	любые файлы	√	√	√	√
`scp` по сети (dropbear)	файлы	USB-Ethernet	√	B/B+	√
UART: `sz` / `rz` (lrzsz)	небольшие файлы	√	√	√	√
TFTP + NFS (глава 3)	ядро и весь rootfs без перепрошивки	—	√	B/B+	√

КАКУЮ ПЛАТУ ПИШЕТ `MAKE FLASH`

Цели `flash` и `flash-boot` работают с **выбранной** прошивкой — из файла `output/.fw`, который пишет `make <fw>_defconfig`. Если вы собирали другую плату через `FW=bbb`, пишите её так же: `make FW=bbb flash DEV=/dev/sdX`. Проверить выбор — `make info`.

Для повседневной разработки программ удобнее всего сеть (`scp` или NFS). Для ядра и загрузчика — `flash-boot` или сетевая загрузка ядра по TFTP.

Цикл разработки

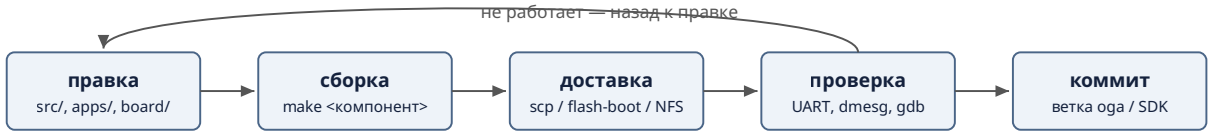


Рис. 1.1. Цикл разработки

Самая дорогая часть цикла — доставка. Главы 2 и 3 посвящены тому, чтобы сократить её с «вынуть карту, прошить, вставить» до одной команды.

ГЛАВА 2

Рабочее место

Консоль UART

UART — главный инструмент: он работает, когда не работает ничего другое. Все платы используют уровни **3,3 В**: адаптер на 5 В может повредить SoC.

Плата	Устройство в Linux	Где контакты	Скорость
OGA	`ttyS2` (UART2)	отладочный UART, к которому подключён адаптер	115200
BBB	`ttyS0` (UART0)	разъём J1: 1 — GND, 4 — RX, 5 — TX	115200
RPi 1	`ttyAMA0` (PL011)	гребёнка: 6 — GND, 8 — TXD (GPIO14), 10 — RXD (GPIO15)	115200
RPi 4/400	`ttyS1` (mini-UART)	те же контакты 6, 8, 10	115200

TX адаптера соединяется с RX платы и наоборот. На ПК:

```
picocom -b 115200 /dev/ttyUSB0          # выход: Ctrl-A Ctrl-X
screen /dev/ttyUSB0 115200              # выход: Ctrl-A K
```

> **ЛОВУШКА: СКОРОСТЬ ПО УМОЛЧАНИЮ**
>
> `screen /dev/ttyUSB0` без скорости открывает порт на 9600 бод. Всегда указывайте `115200`. И не оставляйте отсоединённую сессию `screen`: она держит порт, и другие программы получат «Device or resource busy».

Логин везде — `root` без пароля. На экране (OGA, HDMI у BBB и RPi) тоже есть приглашение на `tty1`.

Сеть

BeagleBone Black, Raspberry Pi 1 B/B+, Raspberry Pi 4/400

На этих платах в rootfs уже настроен DHCP на `eth0` (`BR2_SYSTEM_DHCP="eth0"`) и стоит SSH-сервер dropbear. Подключите кабель к той же сети, что и ПК, и узнайте адрес через UART:

```
# ip -4 addr show eth0
    inet 192.168.1.57/24 ...
```

У Raspberry Pi 1 B/B+ Ethernet висит на USB (чип SMSC LAN9512/9514), у RPi 1 A/A+ и Zero Ethernet нет вовсе — для них подходят USB-Ethernet адаптер (Zero — через OTG-переходник) или UART.

ODROID-GO Advance

Встроенной сети в mainline нет (Wi-Fi ESP8089 Black Edition не поддерживается). Варианты:

1. **USB-Ethernet адаптер** в USB-разъём. Драйверы популярных чипов уже есть в ядре модулями (`CONFIG_USB_RTL8152`, `USB_NET_AX88179_178A`, `USB_NET_AX8817X`, `USB_NET_CDCETHER`), mdev подгрузит модуль при подключении. Интерфейс появится как `eth0`. DHCP у OGA по умолчанию не настроен — поднимите вручную командой `udhcpc -i eth0` или добавьте в `board/oga/buildroot/defconfig` строку `BR2_SYSTEM_DHCP="eth0"` (тогда без адаптера загрузка будет ждать сеть ~15 секунд) и пакет dropbear.
2. **Передача файлов по UART** (раздел ниже).
3. **Картридер**: вынуть карту и скопировать файлы прямо в rootfs.

SSH без пароля

С пустым паролем dropbear не пустит. Два варианта:

- задать пароль на плате: `passwd` (хранится в rootfs и пропадёт после перепрошивки);
- положить свой **публичный** ключ в overlay — тогда доступ будет в каждом образе:

```
mkdir -p board/common/buildroot/rootfs-overlay/root/.ssh
cp ~/.ssh/id_ed25519.pub board/common/buildroot/rootfs-overlay/root/.ssh/authorized_keys
make rootfs image
```

> **ПРАВА**
> Dropbear требует, чтобы `/root/.ssh` и `authorized_keys` не были доступны на запись группе и всем. Overlay копирует права из SDK, поэтому сделайте `chmod 700 .../root/.ssh` и `chmod 600 .../authorized_keys`.

После этого:

```
scp hello root@192.168.1.57:/usr/bin/
ssh root@192.168.1.57 hello
```

Передача файлов по UART

Когда сети нет, небольшие файлы (программы, модули) можно передавать по той же консоли протоколом ZMODEM. Нужен пакет `lrzsz` в rootfs (`BR2_PACKAGE_LRZSZ=y` в defconfig платы) и `lrzsz` + `picocom` на ПК.

```
picocom -b 115200 --send-cmd "sz -vv" --receive-cmd "rz -vv" /dev/ttyUSB0
# на плате:
# cd /tmp && rz
# в picocom: Ctrl-A Ctrl-S, ввести имя файла на ПК
```

Скорость — около 10 КиБ/с на 115200 бод: модуль ядра на 10–20 КиБ приходит за пару секунд, а ядро на 40 МиБ — уже нет.

Картридер

Раздел BOOT — FAT, его монтирует любой ПК. Rootfs — ext4, её тоже можно смонтировать:

```
lsblk                                          # найти карту, пусть это sdX
sudo mount /dev/sdX2 /mnt
sudo cp hello /mnt/usr/bin/ && sync
sudo umount /mnt
```

ОПАСНО

Не перепутайте устройство. И всегда `umount` перед извлечением: ext4 с неотмонтированного раздела может потерять последние записи.

Тулчейн для своих программ

Buildroot уже собрал кросс-компилятор с библиотеками rootfs. Его можно брать прямо из каталога сборки:

Прошивка	Префикс	Где
`oga`	`aarch64-buildroot-linux-gnu-`	`output/oga/buildroot/host/bin/`
`bbb`	`arm-buildroot-linux-gnueabihf-`	`output/bbb/buildroot/host/bin/`
`rpi1`	`arm-buildroot-linux-gnueabihf-`	`output/rpi1/buildroot/host/bin/`
`rpi4`	`aarch64-buildroot-linux-gnu-`	`output/rpi4/buildroot/host/bin/`

```
export PATH=$HOME/linux-sdk/output/bbb/buildroot/host/bin:$PATH
arm-buildroot-linux-gnueabihf-gcc -O2 -o hello hello.c
```

ЛОВУШКА: У BBB И RPI 1 ОДИНАКОВЫЙ ПРЕФИКС, НО РАЗНЫЕ ТУЛЧЕЙНЫ

Оба называются `arm-buildroot-linux-gnueabihf-gcc`, но тулчейн BBB настроен на Cortex-A8 + NEON (ARMv7), а RPi 1 — на ARM1176 (ARMv6). Программа, собранная для BBB, на RPi 1 упадёт с `Illegal instruction`. Следите, чей `host/bin` стоит в `PATH`.

Для работы вне дерева SDK или на другой машине есть переносимый вариант: `make FW=<fw> rootfs-sdk` (первая книга, глава 9). В нём же лежит toolchain-файл для CMake: `share/buildroot/toolchainfile.cmake`.

Обычный системный кросс-компилятор (`aarch64-linux-gnu-gcc` из apt) годится для ядра и U-Boot, но **не для программ**: его glibc и библиотеки отличаются от тех, что в rootfs, и программа может не запуститься с ошибкой про версию glibc.

ГЛАВА 3

Быстрый цикл: загрузка по сети

Перепрошивать карту после каждой правки ядра долго. На платах с Ethernet можно сделать так, чтобы **ядро грузилось по TFTP**, а **rootfs была каталогом на ПК** по NFS. Тогда:

- пересобрали ядро → скопировали в каталог TFTP → перезагрузили плату;
- собрали программу → положили прямо в каталог rootfs на ПК → она сразу видна на плате.

SD-карта при этом нужна только для загрузчика.

Плата	Сеть в U-Boot	Сеть в ядре для NFS-корня
BBB	CPSW (`DRIVER_TI_CPSW`)	`TI_CPSW=y`
RPi 1 B/B+	USB LAN9512/9514 (`USB_ETHER_SMSC95XX`), нужен `usb start`	`USB_NET_SMSC95XX=y`, `USB_DWC2=y`
RPi 4/400	GENET (`BCMGENET`)	`BCMGENET=y`, `BROADCOM_PHY=y` (фрагмент SDK)
OGA	нет	—

> **ЛОВУШКА: ДРАЙВЕР СЕТИ ДЛЯ NFS-КОРНЯ ДОЛЖЕН БЫТЬ ВСТРОЕН**
> Ядро монтирует NFS-корень **до** того, как может загрузить хоть один модуль (модули лежат на том самом корне). Драйвер Ethernet и всё, от чего он зависит, должны быть `=y`. У RPi 4 в arm64 `defconfig` GENET — модуль, поэтому во фрагменте SDK стоит `CONFIG_BCMGENET=y`. Проверка: `grep -E 'BCMGENET|ROOT_NFS|IP_PNP_DHCP' output/rpi4/linux/.config`.

Сервер на ПК

```
sudo apt install tftpd-hpa nfs-kernel-server
```

TFTP

`tftpd-hpa` раздаёт `/srv/tftp`. Сложим туда ядро и dtb каждой прошивки:

```
FW=rpi4
sudo mkdir -p /srv/tftp/$FW
sudo cp output/$FW/linux-install/Image /srv/tftp/$FW/          # у bbb и rpi1 — zImage
sudo cp -r output/$FW/linux-install/dtbs/. /srv/tftp/$FW/      # broadcom/*.dtb
```

NFS

```
sudo mkdir -p /srv/nfs/$FW
sudo tar -xpf output/$FW/images/rootfs.tar -C /srv/nfs/$FW
sudo cp -r output/$FW/linux-install/modules/lib/modules /srv/nfs/$FW/lib/
echo "/srv/nfs/$FW *(rw,sync,no_root_squash,no_subtree_check)" | sudo tee -a /etc/exports
sudo exportfs -ra
```

`no_root_squash` нужен, потому что на плате всё работает от root. Распаковывать rootfs нужно через `sudo`, чтобы файлы получили владельца root.

> **ПОЧЕМУ DHCP ПРИ ЗАГРУЗКЕ НЕ РВЁТ NFS**
> В rootfs сеть поднимается скриптом `S40network` по `/etc/network/interfaces`, где для `eth0` стоит `pre-up /etc/network/nfs_check`. Этот скрипт Buildroot проверяет `/proc/mounts`: если корень смонтирован по NFS через этот интерфейс, он не трогает его. Иначе повторный DHCP мог бы на секунду снять адрес, и система бы зависла.

PXE: тот же extlinux, но по сети

U-Boot умеет PXE: скачать по TFTP файл в формате `extlinux.conf` и загрузить, что в нём написано. Это самый удобный путь: формат уже знаком, а `fdtdir` и автоматический выбор dtb работают так же, как с SD-карты.

Файл `/srv/tftp/pxelinux.cfg/default` (или `01-<mac-адрес-через-дефисы>`, чтобы у каждой платы был свой):

```
default nfs

label nfs
    kernel rpi4/Image
    fdtdir rpi4/
    append root=/dev/nfs nfsroot=192.168.1.10:/srv/nfs/rpi4,v3,tcp ip=dhcp rw console=tty1
console=ttyS1,115200n8
```

`192.168.1.10` — адрес ПК. `ip=dhcp` просит ядро само получить адрес до монтирования корня (опция `IP_PNP_DHCP`, она включена во всех ядрах SDK).

В консоли U-Boot (прервите автозагрузку любой клавишей):

```
=> setenv autoload no          # dhcp только получит адрес, ничего не скачивая
=> dhcp
=> setenv serverip 192.168.1.10
=> pxe get
=> pxe boot
```

Особенности плат:

Плата	Перед dhcp	kernel	fdtdir и ${fdtfile}
BBB	—	`bbb/zImage`	`bbb/` → `bbb/am335x-boneblack.dtb`
RPi 1	`usb start` (Ethernet на USB)	`rpi1/zImage`	`rpi1/` → `rpi1/bcm2835-rpi-b-plus.dtb`
RPi 4/400	—	`rpi4/Image`	`rpi4/` → `rpi4/broadcom/bcm2711-rpi-400.dtb`

В `append` замените консоль: `ttyS0` у BBB, `ttyAMA0` у RPi 1, `ttyS1` у RPi 4.

Ручная загрузка без PXE

Иногда полезно сделать всё по шагам, например чтобы загрузить ядро по сети, а rootfs оставить на карте:

```
=> setenv autoload no; dhcp; setenv serverip 192.168.1.10
=> tftpboot ${kernel_addr_r} bbb/zImage
=> tftpboot ${fdt_addr_r} bbb/${fdtfile}
=> setenv bootargs root=PARTUUID=42424231-02 rootwait rw console=ttyS0,115200n8
=> bootz ${kernel_addr_r} - ${fdt_addr_r}
```

Для arm64 (RPi 4) вместо `bootz` — `booti`. Здесь новое ядро с TFTP работает с rootfs на SD — удобно, когда меняете только ядро. Но помните: модули на SD-карте остались от старого ядра, и если версия или конфигурация изменились, `modprobe` их не загрузит.

Сделать сетевую загрузку постоянной

```
=> setenv bootcmd 'setenv autoload no; dhcp; setenv serverip 192.168.1.10; pxe get && pxe
boot'
=> saveenv
```

`saveenv` у BBB и RPi пишет файл `uboot.env` на раздел BOOT (FAT). Вернуть всё как было: `env default -a; saveenv` в U-Boot или просто удалить `uboot.env` с карты на ПК.

Цикл разработки с NFS

```
make FW=rpi4 linux && sudo cp output/rpi4/linux-install/Image /srv/tftp/rpi4/
# перезагрузить плату — новое ядро

aarch64-buildroot-linux-gnu-gcc -o hello hello.c && sudo cp hello /srv/nfs/rpi4/usr/bin/
# на плате сразу: hello
```

> **СОВЕТ**
>
> Отдайте каталог NFS своему пользователю (`sudo chown -R $USER /srv/nfs/rpi4/root`), чтобы копировать в домашний каталог root на плате без `sudo`.

А если сети нет (OGA)

Для OGA этот цикл недоступен: в U-Boot нет драйвера USB-Ethernet для её адаптера, а своей сети на плате нет. Самые быстрые варианты:

- программы — USB-Ethernet адаптер в работающем Linux и `scp` (глава 2);
- ядро и dtb — `make linux image` + `make flash-boot DEV=/dev/sdX` через картридер (rootfs и ваши файлы на ней не трогаются);
- небольшие файлы — UART и `rz`.

ГЛАВА 4

Свои программы

Самая короткая дорога

docs/dev-book/examples/app/hello/hello.c

```c
/* Minimal program: prints where it runs. */
#include <stdio.h>
#include <sys/utsname.h>

int main(void)
{
    struct utsname u;

    if (uname(&u) != 0) {
        perror("uname");
        return 1;
    }
    printf("Hello from %s: %s, kernel %s\n", u.nodename, u.machine, u.release);
    return 0;
}
```

```
SDK=~/linux-sdk; FW=oga
export PATH=$SDK/output/$FW/buildroot/host/bin:$PATH
aarch64-buildroot-linux-gnu-gcc -O2 -g -Wall -o hello hello.c
file hello      # ELF 64-bit LSB pie executable, ARM aarch64 ...
scp hello root@board:/tmp/ && ssh root@board /tmp/hello
```

На плате:

```
Hello from odroid-go: aarch64, kernel 7.2.9-oga
```

Makefile-проект

Свой Makefile пишите так, чтобы компилятор и флаги **можно было переопределить** снаружи (`?=`), а установка шла в `$(DESTDIR)`. Тогда один и тот же Makefile работает и для сборки руками, и внутри пакета Buildroot (глава 6):

docs/dev-book/examples/app/hello/Makefile

```
# Cross-compile: make CC=<sdk>/bin/aarch64-buildroot-linux-gnu-gcc
CC     ?= gcc
CFLAGS ?= -O2 -g -Wall -Wextra

hello: hello.c
    $(CC) $(CFLAGS) -o $@ $<

install: hello
    install -D -m 0755 hello $(DESTDIR)/usr/bin/hello

clean:
    rm -f hello

.PHONY: install clean
```

```
make CC=aarch64-buildroot-linux-gnu-gcc
make CC=aarch64-buildroot-linux-gnu-gcc DESTDIR=/srv/nfs/rpi4 install   # прямо в NFS-корень
```

CMake-проект

docs/dev-book/examples/app/cmake-demo/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(cmake_demo C)

add_executable(cmake-demo main.c)
target_compile_options(cmake-demo PRIVATE -Wall -Wextra)
install(TARGETS cmake-demo DESTINATION bin)
```

Buildroot кладёт готовый toolchain-файл для CMake в `host/share/buildroot/`:

```
cmake -S cmake-demo -B build-oga \
      -DCMAKE_TOOLCHAIN_FILE=$SDK/output/oga/buildroot/host/share/buildroot/
toolchainfile.cmake
cmake --build build-oga
```

Toolchain-файл задаёт компилятор, sysroot и флаги процессора (`-mcpu=cortex-a35` и т. п.), так что `find_package()` и `pkg-config` ищут библиотеки в sysroot прошивки, а не на ПК.

Библиотеки

Программа может использовать только те библиотеки, которые **есть в rootfs**. Если нужна библиотека, которой нет (например, `libgpiod2`, `sdl2`, `libevdev`), сначала включите её в Buildroot (глава 6): после `make rootfs` её заголовки и `.so` появятся в sysroot (`output/<fw>/buildroot/staging/`), а `.so` — ещё и в rootfs.

Флаги для компиляции и линковки даёт `pkg-config` из того же `host/bin`. Это обёртка Buildroot, которая уже знает про sysroot:

```
aarch64-buildroot-linux-gnu-gcc app.c -o app $(pkg-config --cflags --libs libgpiod)
```

> **ЛОВУШКА: СИСТЕМНЫЙ PKG-CONFIG**
> Если в `PATH` раньше стоит системный `/usr/bin/pkg-config`, он вернёт пути к библиотекам ПК (`-I/usr/include`), и сборка либо упадёт, либо — хуже — соберётся с чужими заголовками. Проверяйте: `which pkg-config` должен указывать в `host/bin`.

Отладка

gdbserver по сети

В rootfs есть `gdbserver` (`BR2_PACKAGE_GDB=y`), а на ПК — `<префикс>-gdb` (`BR2_PACKAGE_HOST_GDB=y`), собранный под эту архитектуру.

```
# плата
gdbserver :2345 /tmp/hello

# ПК
aarch64-buildroot-linux-gnu-gdb hello
(gdb) set sysroot /home/you/linux-sdk/output/oga/buildroot/staging
(gdb) target remote 192.168.1.57:2345
(gdb) break main
(gdb) continue
```

`set sysroot` нужен, чтобы gdb нашёл на ПК те же библиотеки (`libc.so.6` и др.), что загружены на плате, — с отладочной информацией.

gdbserver по UART (OGA)

У OGA нет сети, зато на разъёме расширения P2 выведен второй UART (`ttyS1`, контакты 2 — RXD, 3 — TXD). gdbserver умеет работать через последовательный порт:

```
# плата (ttyS1 — разъём P2; не консольный ttyS2!)
gdbserver /dev/ttyS1 /tmp/hello

# ПК, второй USB-UART адаптер на P2
aarch64-buildroot-linux-gnu-gdb hello
(gdb) set serial baud 115200
(gdb) target remote /dev/ttyUSB1
```

> **СКОРОСТЬ TTYS1**
> gdbserver не меняет скорость порта. Выставьте её заранее: `stty -F /dev/ttyS1 115200 raw`.

strace и журнал

- `strace` (пакет `BR2_PACKAGE_STRACE`) показывает системные вызовы: незаменим, когда программа «молча» не работает (`strace -f ./app 2>&1 | grep -E 'ENOENT|EACCES'`).
- Вывод в системный журнал — `logger -t myapp "текст"`, читать — `/var/log/messages` (busybox syslogd пишет в файл; `logread` в конфигурации busybox Buildroot выключен).
- Сообщения ядра — `dmesg`, по ходу работы — `dmesg -w`.

Что важно помнить

- Собирайте **тулчейном прошивки**, а не системным: у них разные glibc.
- Не путайте `host/bin` BBB и RPi 1: префикс одинаковый, ISA разная.
- Отладочная информация (`-g`) не мешает: для rootfs Buildroot сам вырезает её при установке пакета, а при копировании руками можно сделать `<префикс>-strip hello`.

ГЛАВА 5

Программы для конкретных плат

Ядро даёт к железу стандартные интерфейсы: `/dev/input/event*` для кнопок, `/dev/fb0` для экрана, `/dev/gpiochipN` для GPIO, `/sys/class/leds/` для светодиодов, `/dev/ttyS*` для UART, ALSA для звука. Ваша программа работает с ними одинаково на любой плате — отличаются только имена устройств и номера линий.

Светодиоды: без единой строки кода

Каждый светодиод из device tree — каталог в `/sys/class/leds/`:

Плата	Светодиоды
OGA	`blue:status` (по умолчанию `heartbeat`), `red:charging`
BBB	`beaglebone:green:heartbeat`, `…:mmc0`, `…:usr2`, `…:usr3`
RPi 1 B+/A+, RPi 4 B	`ACT`, `PWR`
RPi 400	только `PWR` (`led_act` удалён в DTS платы)

```
# cd /sys/class/leds/blue:status           (на BBB: beaglebone:green:usr3, на RPi: ACT)
# cat trigger                              [heartbeat] none timer default-on mmc0 ...
# echo none > trigger
# echo 255 > brightness ; echo 0 > brightness
# echo timer > trigger ; echo 100 > delay_on ; echo 900 > delay_off
```

«Триггеры» — это готовые источники событий в ядре: сердцебиение, активность SD, таймер, нагрузка CPU. Для многих задач индикации свой код вообще не нужен.

GPIO: BeagleBone Black и Raspberry Pi

Старый интерфейс `/sys/class/gpio/export` устарел. Современный — **символьное устройство** `/dev/gpiochipN` (опция `GPIO_CDEV`, включена во всех ядрах SDK). С ним работают утилиты и библиотека **libgpiod**, а можно и напрямую через `ioctl`.

Утилиты libgpiod

Включите в defconfig платы `BR2_PACKAGE_LIBGPIOD2=y` и `BR2_PACKAGE_LIBGPIOD2_TOOLS=y` (глава 6), пересоберите rootfs. На плате:

```
# gpiodetect                           какие есть контроллеры (вывод на RPi 4, примерный)
gpiochip0 [pinctrl-bcm2711] (58 lines)
# gpioinfo gpiochip0 | head            линии и кто их занял
# gpioset -c gpiochip0 17=1            включить GPIO17
```

Своя программа

docs/dev-book/examples/app/gpio-blink/gpio-blink.c

```
/*
 * Blink one GPIO line through the GPIO character device (uAPI v2).
 *
```

```
 *   gpio-blink <chip> <line> [count] [period_ms]
 *   gpio-blink /dev/gpiochip0 17 10 500      Raspberry Pi: GPIO17, header pin 11
 *   gpio-blink /dev/gpiochip1 28 10 500      BeagleBone Black: GPIO1_28, P9_12
 *
 * Which /dev/gpiochipN is which bank: "gpiodetect" or
 * /sys/bus/gpio/devices/gpiochipN/label.
 */
#include <errno.h>
#include <fcntl.h>
#include <linux/gpio.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    struct gpio_v2_line_request req;
    struct gpio_v2_line_values val;
    int fd, count, period, i;

    if (argc < 3) {
        fprintf(stderr, "usage: %s <chip> <line> [count] [period_ms]\n", argv[0]);
        return 2;
    }
    count = argc > 3 ? atoi(argv[3]) : 10;
    period = argc > 4 ? atoi(argv[4]) : 500;

    fd = open(argv[1], O_RDONLY | O_CLOEXEC);
    if (fd < 0) {
        perror(argv[1]);
        return 1;
    }

    memset(&req, 0, sizeof(req));
    req.offsets[0] = atoi(argv[2]);
    req.num_lines = 1;
    req.config.flags = GPIO_V2_LINE_FLAG_OUTPUT;
    strncpy(req.consumer, "gpio-blink", sizeof(req.consumer) - 1);
    if (ioctl(fd, GPIO_V2_GET_LINE_IOCTL, &req) < 0) {
        fprintf(stderr, "line %s: %s\n", argv[2], strerror(errno));
        return 1;
    }
    close(fd);

    val.mask = 1;
    for (i = 0; i < count * 2; i++) {
        val.bits = !(i & 1);
        if (ioctl(req.fd, GPIO_V2_LINE_SET_VALUES_IOCTL, &val) < 0) {
            perror("set value");
            return 1;
        }
        usleep(period * 1000 / 2);
    }
    close(req.fd);      /* the line is released, the pin keeps its last value */
    return 0;
}
```

Программа использует только заголовок ядра `linux/gpio.h` (uAPI v2), так что никаких библиотек в rootfs не требует.

Плата	Контакт	Контроллер и линия	Команда
RPi 1 / 4 / 400	гребёнка, контакт 11	GPIO17	`gpio-blink /dev/gpiochip0 17`
BBB	P9, контакт 12	GPIO1_28: банк 1, линия 28	`gpio-blink /dev/gpiochipN 28`

Схема для проверки: светодиод через резистор 330–1000 Ом между контактом и GND (у RPi GND — контакт 9, у BBB — P9.1/P9.2).

BBB: НОМЕР КОНТРОЛЛЕРА И PINMUX

1. Банку 1 не обязательно соответствует `/dev/gpiochip1`: порядок регистрации контроллеров может отличаться. Ищите по адресу банка (`4804c000` у GPIO1) в выводе `gpiodetect`.
2. Каждый вывод AM335x может выполнять несколько функций, и функцию выбирает **pinmux**. Если вывод не настроен как GPIO (mode 7), программа отработает без ошибок, а на контакте ничего не изменится. Надёжный способ — описать pinmux в device tree (overlay в главе 8 так и делает).

RPI: ФУНКЦИЯ ВЫВОДА

У BCM2835/2711 контроллер выводов сам переключает вывод в режим GPIO, когда линию запрашивают через `/dev/gpiochip0`. Отдельный pinmux не нужен.

ODROID-GO Advance: кнопки и экран

У OGA нет свободных GPIO, зато есть геймпад и экран. Пример `oga-pad` двигает квадрат по экрану крестовиной:

docs/dev-book/examples/app/oga-pad/oga-pad.c

```
/*
 * oga-pad: move a square across the screen with the D-pad (ODROID-GO Advance).
 *
 * Shows three things: finding an input device by name (evdev), drawing on
 * the framebuffer (/dev/fb0, DRM fbdev emulation) and switching the text
 * console off while drawing (KD_GRAPHICS), so fbcon does not draw over us.
 *
 * The OGA panel is mounted rotated: the framebuffer keeps the panel's native
 * (portrait) orientation, and only the text console is turned by fbcon
 * (fbcon=rotate:N, needs CONFIG_FRAMEBUFFER_CONSOLE_ROTATION). The program
 * works in "screen" coordinates as the user sees them and maps every pixel
 * the same way fbcon does. The rotation is taken from
 * /sys/class/graphics/fbcon/rotate; "oga-pad N" overrides it.
 *
 * A/B/X/Y change the colour, F6 (BTN_TRIGGER_HAPPY6) or Ctrl-C quits.
 */
#include <fcntl.h>
#include <linux/fb.h>
#include <linux/input.h>
#include <linux/kd.h>
#include <poll.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <unistd.h>

#define SIZE 40
#define STEP 20

static volatile sig_atomic_t quit;
static uint8_t *fb;
static struct fb_var_screeninfo var;
static struct fb_fix_screeninfo fix;
static int rotate;      /* like fbcon=rotate: 0, 1 = 90 cw, 2 = 180, 3 = 270 cw */
static int scr_w, scr_h;    /* screen size as the user sees it */
```

```
/* Current text console rotation, 0 if fbcon rotation is not available */
static int fbcon_rotation(void)
{
    FILE *f = fopen("/sys/class/graphics/fbcon/rotate", "r");
    int r = 0;

    if (f) {
        if (fscanf(f, "%d", &r) != 1)
            r = 0;
        fclose(f);
    }
    return r & 3;
}

static void on_signal(int sig)
{
    (void)sig;
    quit = 1;
}

/* Open /dev/input/eventN whose device name is 'want' ("gpio-keys" on OGA) */
static int open_input(const char *want)
{
    char path[32], name[64];
    int i, fd;

    for (i = 0; i < 32; i++) {
        snprintf(path, sizeof(path), "/dev/input/event%d", i);
        fd = open(path, O_RDONLY | O_CLOEXEC);
        if (fd < 0)
            continue;
        memset(name, 0, sizeof(name));
        if (ioctl(fd, EVIOCGNAME(sizeof(name) - 1), name) >= 0 && !strcmp(name, want))
            return fd;
        close(fd);
    }
    return -1;
}

static void put_pixel(int sx, int sy, uint32_t rgb)
{
    int x, y, bpp = var.bits_per_pixel / 8;
    uint8_t *p;

    switch (rotate) {   /* screen -> framebuffer coordinates */
    case 1:  x = var.xres - 1 - sy; y = sx; break;
    case 2:  x = var.xres - 1 - sx; y = var.yres - 1 - sy; break;
    case 3:  x = sy; y = var.yres - 1 - sx; break;
    default: x = sx; y = sy; break;
    }
    p = fb + (size_t)y * fix.line_length + (size_t)x * bpp;
    if (bpp == 4)
        *(uint32_t *)p = rgb;
    else if (bpp == 2)  /* RGB565 */
        *(uint16_t *)p = ((rgb >> 8) & 0xf800) | ((rgb >> 5) & 0x07e0) | ((rgb >> 3) &
0x001f);
}

static void fill(int x0, int y0, int w, int h, uint32_t rgb)
{
    int x, y;

    for (y = y0; y < y0 + h && y < scr_h; y++)
        for (x = x0; x < x0 + w && x < scr_w; x++)
            put_pixel(x, y, rgb);
}

int main(int argc, char **argv)
{
    uint32_t colour = 0xffffff;
    int px, py, fbfd, in, tty;
```

```
    struct input_event ev;
    struct pollfd pfd;

    in = open_input("gpio-keys");
    if (in < 0) {
        fputs("gpio-keys input device not found\n", stderr);
        return 1;
    }
    fbfd = open("/dev/fb0", O_RDWR | O_CLOEXEC);
    if (fbfd < 0 || ioctl(fbfd, FBIOGET_VSCREENINFO, &var) < 0 ||
        ioctl(fbfd, FBIOGET_FSCREENINFO, &fix) < 0) {
        perror("/dev/fb0");
        return 1;
    }
    fb = mmap(NULL, fix.smem_len, PROT_READ | PROT_WRITE, MAP_SHARED, fbfd, 0);
    if (fb == MAP_FAILED) {
        perror("mmap");
        return 1;
    }
    rotate = argc > 1 ? atoi(argv[1]) & 3 : fbcon_rotation();
    scr_w = rotate & 1 ? var.yres : var.xres;
    scr_h = rotate & 1 ? var.xres : var.yres;
    printf("fb0: %ux%u, %u bpp; screen %dx%d (rotate %d)\n", var.xres, var.yres,
           var.bits_per_pixel, scr_w, scr_h, rotate);

    /* Keep fbcon (the blinking cursor) away from our pixels */
    tty = open("/dev/tty1", O_RDWR | O_CLOEXEC);
    if (tty >= 0)
        ioctl(tty, KDSETMODE, KD_GRAPHICS);
    signal(SIGINT, on_signal);
    signal(SIGTERM, on_signal);

    px = (scr_w - SIZE) / 2;
    py = (scr_h - SIZE) / 2;
    fill(0, 0, scr_w, scr_h, 0);
    fill(px, py, SIZE, SIZE, colour);

    pfd.fd = in;
    pfd.events = POLLIN;
    while (!quit) {
        if (poll(&pfd, 1, 200) <= 0)
            continue;
        if (read(in, &ev, sizeof(ev)) != sizeof(ev) || ev.type != EV_KEY || ev.value != 1)
            continue;

        fill(px, py, SIZE, SIZE, 0);
        switch (ev.code) {
        case BTN_DPAD_UP:    py -= STEP; break;
        case BTN_DPAD_DOWN:  py += STEP; break;
        case BTN_DPAD_LEFT:  px -= STEP; break;
        case BTN_DPAD_RIGHT: px += STEP; break;
        case BTN_SOUTH:      colour = 0xff3030; break;
        case BTN_EAST:       colour = 0x30ff30; break;
        case BTN_WEST:       colour = 0x3080ff; break;
        case BTN_NORTH:      colour = 0xffff30; break;
        case BTN_TRIGGER_HAPPY6: quit = 1; break;
        }
        if (px < 0) px = 0;
        if (py < 0) py = 0;
        if (px > scr_w - SIZE) px = scr_w - SIZE;
        if (py > scr_h - SIZE) py = scr_h - SIZE;
        fill(px, py, SIZE, SIZE, colour);
    }

    if (tty >= 0)
        ioctl(tty, KDSETMODE, KD_TEXT);
    munmap(fb, fix.smem_len);
    return 0;
}
```

Три вещи, которые стоит из него вынести.

Поиск устройства ввода по имени. Номер `eventN` зависит от порядка инициализации драйверов. Надёжнее перебрать `/dev/input/event*` и спросить имя через `EVIOCGNAME`: у кнопок OGA это `gpio-keys`, у стика — `adc-joystick`.

Коды кнопок. Они заданы в DTS (`rk3326-odroid-go.dtsi`): крестовина — `BTN_DPAD_*`, A/B/X/Y — `BTN_EAST`/`BTN_SOUTH`/`BTN_WEST`/`BTN_NORTH`, F1–F6 — `BTN_TRIGGER_HAPPY1…6`, курки — `BTN_TL`/`BTN_TR`. Стик — оси `ABS_X`/`ABS_Y` (события `EV_ABS`). Посмотреть всё вживую: `evtest`.

Поворот экрана. Панель OGA установлена повёрнутой. Ядро поворачивает только текстовую консоль (`fbcon=rotate:3` в командной строке; работает, только если в ядре включён `CONFIG_FRAMEBUFFER_CONSOLE_ROTATION`), а `/dev/fb0` остаётся в родной портретной ориентации панели. Программа читает текущий поворот консоли из `/sys/class/graphics/fbcon/rotate`, работает в «экранных» координатах и пересчитывает каждую точку так же, как fbcon. Ещё одна деталь — `KDSETMODE KD_GRAPHICS` на `/dev/tty1`: без него мигающий курсор консоли будет рисоваться поверх картинки.

```
# oga-pad                          (примерный вывод: панель 320×480)
fb0: 320x480, 32 bpp; screen 480x320 (rotate 3)
```

Поворот можно задать и вручную: `oga-pad 1`. Значения те же, что у `fbcon=rotate:` — 0, 1 (90° по часовой), 2, 3 (90° против часовой). Программа поддерживает 32 и 16 бит на точку.

> **ДЛЯ НАСТОЯЩИХ ИГР**
> Для графики с ускорением используйте SDL2 (`BR2_PACKAGE_SDL2`, бэкенд KMS/DRM). SDL сам найдёт геймпад и экран, но поворот панели всё равно придётся учитывать.

ODROID-GO Advance: UART на разъёме

На разъёме расширения P2 выведен UART1: контакт 2 — RXD, контакт 3 — TXD (так написано в комментарии DTS платы). В Linux это `/dev/ttyS1`. К нему можно подключить GPS-модуль, микроконтроллер или второй USB-UART адаптер:

docs/dev-book/examples/app/serial-echo/serial-echo.c

```
/*
 * serial-echo <tty> [baud]: echo every received line back, prefixed.
 *
 * ODROID-GO Advance: /dev/ttyS1 is UART1 on the expansion header P2
 * (pin 2 RXD, pin 3 TXD). Do not use the console UART (ttyS2): getty owns it.
 */
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

static speed_t to_speed(int baud)
{
    switch (baud) {
    case 9600:   return B9600;
    case 57600:  return B57600;
    case 115200: return B115200;
    case 921600: return B921600;
    default:     return 0;
    }
}
```

```
int main(int argc, char **argv)
{
    char line[256], c;
    struct termios t;
    size_t len = 0;
    speed_t speed;
    int fd;

    if (argc < 2) {
        fprintf(stderr, "usage: %s <tty> [baud]\n", argv[0]);
        return 2;
    }
    speed = to_speed(argc > 2 ? atoi(argv[2]) : 115200);
    if (!speed) {
        fputs("unsupported baud rate\n", stderr);
        return 2;
    }
    fd = open(argv[1], O_RDWR | O_NOCTTY | O_CLOEXEC);
    if (fd < 0 || tcgetattr(fd, &t) < 0) {
        perror(argv[1]);
        return 1;
    }
    cfmakeraw(&t);          /* 8N1, no echo, no line editing */
    t.c_cflag |= CLOCAL | CREAD;
    t.c_cc[VMIN] = 1;
    t.c_cc[VTIME] = 0;
    cfsetispeed(&t, speed);
    cfsetospeed(&t, speed);
    if (tcsetattr(fd, TCSANOW, &t) < 0) {
        perror("tcsetattr");
        return 1;
    }

    while (read(fd, &c, 1) == 1) {
        if (c == '\r')
            continue;
        if (c != '\n' && len < sizeof(line) - 1) {
            line[len++] = c;
            continue;
        }
        line[len] = '\0';
        dprintf(fd, "echo: %s\r\n", line);
        printf("rx: %s\n", line);
        len = 0;
    }
    return 0;
}
```

```
# плата
serial-echo /dev/ttyS1 115200
# ПК, адаптер на P2
picocom -b 115200 --omap crlf /dev/ttyUSB1      # набираете строку — получаете "echo: ..."
```

`cfmakeraw()` переводит порт в «сырой» режим: без эха, без построчного редактирования и без преобразования `\r`/`\n`. Почти всегда это то, что нужно для обмена с устройствами.

Звук

ALSA одинакова на всех платах, отличаются карты: RK817 на OGA, HDMI у RPi 4 и BBB.

```
# aplay -l                          список карт
# amixer                            регуляторы громкости
# speaker-test -t sine -f 440 -l 1
```

На OGA звук выводится на встроенный динамик или в наушники; переключение — регулятором `amixer` (смотрите список `amixer controls`).

ГЛАВА 6

Программы в rootfs: пакеты Buildroot

Копировать программы на плату руками удобно, пока идёт разработка. Но всё, что должно быть **в каждом образе**, нужно оформить так, чтобы `make rootfs image` собирал это само. Для этого есть три механизма: готовые пакеты Buildroot, свои пакеты и overlay.

Включить готовый пакет

В Buildroot почти 3000 пакетов. Найти нужный:

```
ls src/buildroot/package | grep -i gpio
make FW=bbb rootfs-menuconfig          # меню; поиск — клавиша /
```

После выбора в меню сохраните изменения обратно в defconfig платы:

```
make FW=bbb rootfs-savedefconfig       # пишет board/bbb/buildroot/defconfig
git diff board/bbb/buildroot/defconfig # проверить, что добавилось
make FW=bbb rootfs image
```

Можно и сразу дописать строки в `board/<плата>/buildroot/defconfig`: SDK заметит, что файл новее `.config`, и переконфигурирует Buildroot сам.

Полезные пакеты по платам:

Пакет	Опция	Для чего	Платы
libgpiod2 + утилиты	`BR2_PACKAGE_LIBGPIOD2`, `_TOOLS`	GPIO из программ и shell	BBB, RPi
i2c-tools	`BR2_PACKAGE_I2C_TOOLS`	`i2cdetect`, `i2cget`	все
strace	`BR2_PACKAGE_STRACE`	отладка системных вызовов	все
lrzsz	`BR2_PACKAGE_LRZSZ`	файлы по UART	все, особенно OGA
dropbear	`BR2_PACKAGE_DROPBEAR`	SSH	OGA (у остальных уже есть)
SDL2	`BR2_PACKAGE_SDL2`, `_KMSDRM`	игры и графика	OGA, RPi 4
libevdev	`BR2_PACKAGE_LIBEVDEV`	удобная работа с вводом	OGA
picocom	`BR2_PACKAGE_PICOCOM`	терминал для UART на плате	все

ЛОВУШКА: ВКЛЮЧИЛИ ПАКЕТ — А УЖЕ СОБРАННЫЙ НЕ ПЕРЕСОБРАЛСЯ

Если пакет **уже собран**, а вы поменяли его опции (например, включили `BR2_PACKAGE_SDL2_KMSDRM`), Buildroot его не пересоберёт. Нужно `make rootfs-make BR=sdl2-reconfigure` (или `-dirclean`, а потом `make rootfs`). Новые пакеты собираются автоматически.

Свой пакет

Возьмём программу `hello` из главы 4 и сделаем так, чтобы она попадала в каждый образ вместе с init-скриптом, который запускает её при загрузке.

Где что лежит

```
apps/hello/                                  исходники (в SDK)
  hello.c, Makefile
board/common/buildroot/package/sdk-hello/    описание пакета
  Config.in        пункт меню
  sdk-hello.mk     как собирать и устанавливать
  S90sdk-hello     init-скрипт
board/common/buildroot/Config.in             подключение пункта меню
```

Исходники можно держать и в отдельном git-репозитории — тогда пакет будет скачивать их (`SDK_HELLO_SITE = https://...git`, `SDK_HELLO_SITE_METHOD = git`). Для своих программ в активной разработке удобнее локальный каталог.

Config.in

docs/dev-book/examples/package/sdk-hello/Config.in

```
config BR2_PACKAGE_SDK_HELLO
    bool "sdk-hello"
    help
      OdroidLinux SDK example: a small C program built from a
      local source directory (apps/hello in the SDK), plus an init
      script that runs it at boot.
```

И строка в общем `board/common/buildroot/Config.in`, чтобы пункт появился в меню:

```
source "$BR2_EXTERNAL_SDK_PATH/package/sdk-hello/Config.in"
```

sdk-hello.mk

docs/dev-book/examples/package/sdk-hello/sdk-hello.mk

```
################################################################################
#
# sdk-hello
#
################################################################################

SDK_HELLO_VERSION = 1.0
# Source lives in the SDK itself: <sdk>/apps/hello
SDK_HELLO_SITE = $(BR2_EXTERNAL_SDK_PATH)/../../../apps/hello
SDK_HELLO_SITE_METHOD = local
SDK_HELLO_LICENSE = GPL-2.0

define SDK_HELLO_BUILD_CMDS
    $(TARGET_MAKE_ENV) $(MAKE) -C $(@D) $(TARGET_CONFIGURE_OPTS)
endef

define SDK_HELLO_INSTALL_TARGET_CMDS
    $(TARGET_MAKE_ENV) $(MAKE) -C $(@D) DESTDIR=$(TARGET_DIR) install
endef

define SDK_HELLO_INSTALL_INIT_SYSV
    $(INSTALL) -D -m 0755 $(SDK_HELLO_PKGDIR)/S90sdk-hello \
        $(TARGET_DIR)/etc/init.d/S90sdk-hello
endef
```

```
$(eval $(generic-package))
```

Разберём:

- Имя переменных — имя пакета заглавными, `-` заменяется на `_`: `sdk-hello` → `SDK_HELLO_*`.
- `SITE_METHOD = local` — исходники берутся из каталога (копируются через `rsync` в `output/<fw>/buildroot/build/sdk-hello-1.0/`).
- `$(BR2_EXTERNAL_SDK_PATH)` — путь к `board/common/buildroot`, поэтому `../../../apps/hello` указывает на `apps/hello` в корне SDK.
- `TARGET_CONFIGURE_OPTS` передаёт `CC`, `CFLAGS`, `LDFLAGS` и прочее для тулчейна прошивки. Наш Makefile задаёт их через `?=`, поэтому значения Buildroot побеждают.
- `$(@D)` — каталог сборки пакета, `$(TARGET_DIR)` — будущая rootfs.
- `INSTALL_INIT_SYSV` — установка init-скрипта для busybox init (а для systemd был бы `INSTALL_INIT_SYSTEMD`).
- `$(eval $(generic-package))` — «generic» означает «я сам скажу, как собирать». Для CMake есть `cmake-package`, для autotools — `autotools-package`, для Meson — `meson-package`: им достаточно `SITE` и пары опций.

Init-скрипт

docs/dev-book/examples/package/sdk-hello/S90sdk-hello

```
#!/bin/sh
# Runs /usr/bin/hello once at boot and logs the result (sdk-hello package).

case "$1" in
start)
    printf "Starting sdk-hello: "
    /usr/bin/hello | logger -t sdk-hello && echo "OK" || echo "FAIL"
    ;;
stop|restart|reload)
    ;;
*)
    echo "Usage: $0 {start|stop|restart}"
    exit 1
    ;;
esac
```

Busybox init запускает `/etc/init.d/S??*` по порядку имён с аргументом `start` при загрузке и `stop` при выключении. Номер задаёт порядок: `S10mdev` (устройства), `S40network` (сеть), ваши службы — после сети, например `S90`.

Включить и собрать

```
echo 'BR2_PACKAGE_SDK_HELLO=y' >> board/oga/buildroot/defconfig
make FW=oga rootfs image
```

Проверить без железа:

```
tar -tvf output/oga/images/rootfs.tar | grep -E 'usr/bin/hello|S90sdk-hello'
```

Изменили исходник — что пересобрать

> **ЛОВУШКА: `make rootfs` НЕ ВИДИТ ИЗМЕНЕНИЙ В ИСХОДНИКАХ**
>
> Пакет с `SITE_METHOD = local` копирует исходники **один раз** при первой сборке. Если поменять `apps/hello/hello.c` и запустить `make rootfs`, Buildroot решит, что пакет уже собран, и в rootfs останется **старая** программа. Это проверено: после правки строки в `hello.c` обычный `make` оставил старый бинарник, а `sdk-hello-rebuild` — подхватил изменение.

```
make FW=oga rootfs-make BR=sdk-hello-rebuild    # rsync исходников + сборка + установка
make FW=oga rootfs image
```

Тот же приём для чужих пакетов: OVERRIDE_SRCDIR

Если нужно поправить пакет из Buildroot (busybox, SDL2…), не трогая архив, укажите Buildroot свой каталог с исходниками. Файл `output/<fw>/buildroot/local.mk`:

```
BUSYBOX_OVERRIDE_SRCDIR = /home/you/src/busybox
```

Теперь `busybox-rebuild` будет брать код оттуда. Это локальная настройка вашей сборки: файл лежит в `output/` и в git не попадает.

Overlay: просто файлы

Конфигурационные файлы, скрипты, ключи — то, что не надо компилировать, — кладите в `board/common/buildroot/rootfs-overlay/` с той же структурой каталогов, что в rootfs:

```
board/common/buildroot/rootfs-overlay/
├── etc/profile.d/sdk.sh          уже есть: приглашение shell
├── etc/init.d/S95myservice       свой init-скрипт (не забудьте chmod +x)
└── root/.ssh/authorized_keys     ключ для SSH (глава 2)
```

Overlay общий для всех плат. Если файл нужен только одной плате — сделайте свой overlay `board/<плата>/buildroot/rootfs-overlay` и добавьте его в `BR2_ROOTFS_OVERLAY` defconfig платы через пробел.

> **ЛОВУШКА: УДАЛЁННЫЕ ФАЙЛЫ ОСТАЮТСЯ**
>
> Buildroot никогда не удаляет из `output/<fw>/buildroot/target/` то, что туда уже скопировал. Переименовали или удалили файл в overlay — удалите его и из `target/` (или `make rootfs-clean rootfs`).

Быстрая проверка пакета отдельно

Полная сборка rootfs долгая. Пакет можно проверить в отдельной маленькой сборке Buildroot, которая использует **уже собранный тулчейн** прошивки как внешний:

```
BR2_aarch64=y
BR2_cortex_a35=y
BR2_TOOLCHAIN_EXTERNAL=y
BR2_TOOLCHAIN_EXTERNAL_CUSTOM=y
BR2_TOOLCHAIN_EXTERNAL_PREINSTALLED=y
BR2_TOOLCHAIN_EXTERNAL_PATH="/home/you/linux-sdk/output/oga/buildroot/host"
```

```
BR2_TOOLCHAIN_EXTERNAL_CUSTOM_PREFIX="aarch64-buildroot-linux-gnu"
BR2_TOOLCHAIN_EXTERNAL_GCC_15=y
BR2_TOOLCHAIN_EXTERNAL_HEADERS_7_1=y
BR2_TOOLCHAIN_EXTERNAL_CUSTOM_GLIBC=y
BR2_TOOLCHAIN_EXTERNAL_CXX=y
# BR2_TOOLCHAIN_EXTERNAL_INET_RPC is not set
BR2_PACKAGE_SDK_HELLO=y
```

```
make -C src/buildroot O=/tmp/br-test BR2_EXTERNAL=$PWD/board/common/buildroot \
     defconfig BR2_DEFCONFIG=/tmp/br-test.defconfig
make -C src/buildroot O=/tmp/br-test sdk-hello
ls -l /tmp/br-test/target/usr/bin/hello
```

Именно так проверялся пакет из этой главы: сборка заняла меньше минуты. Версии GCC и заголовков ядра в defconfig должны совпадать с тулчейном (для SDK — GCC 15 и заголовки 7.1), а строка про `INET_RPC` нужна, потому что в glibc тулчейна нет RPC.

ГЛАВА 7

Модули ядра вне дерева

Модуль ядра — код, который загружается в работающее ядро (`insmod`/`modprobe`) и работает в его адресном пространстве с полными правами. Модуль «вне дерева» (out-of-tree) живёт в своём каталоге, а не в `src/linux`. Это лучший способ начать: не нужно пересобирать ядро, цикл «правка — проверка» занимает секунды.

Главное правило: тот же самый ядро

Модуль собирается **против конкретного собранного ядра** — того, что лежит в `output/<fw>/linux`. Оттуда берутся конфигурация, заголовки, сгенерированные файлы и таблица экспортируемых символов (`Module.symvers`). При загрузке ядро сверяет «vermagic» модуля со своей:

```
$ modinfo -F vermagic hello.ko
7.2.9-oga SMP preempt mod_unload aarch64           OGA
7.2.9-bbb SMP mod_unload modversions ARMv6 p2v8    BBB
7.2.9-rpi1 mod_unload ARMv6 p2v8                   RPi 1
7.2.9-rpi4 SMP preempt mod_unload aarch64          RPi 4
```

Если версия или ключевые опции не совпадают, `insmod` откажется: `Invalid module format`. Отсюда правило: **пересобрали ядро — пересоберите модули**.

> **ОСОБЕННОСТИ ПО ПЛАТАМ**
>
> - **BBB**: в `omap2plus_defconfig` включён `MODVERSIONS` — каждая используемая функция ядра проверяется по контрольной сумме. Модуль от «почти такого же» ядра не загрузится.
> - **BBB, RPi 1**: в vermagic стоит `ARMv6`. У RPi 1 это родная архитектура, а у BBB — следствие того, что `omap2plus_defconfig` поддерживает и старые OMAP2 (ARMv6), поэтому ядро собрано с базовым набором инструкций ARMv6.
> - **RPi 1**: одноядерный, без `SMP` и `preempt` — ошибки синхронизации, которые на многоядерных платах всплывают сразу, здесь могут годами не проявляться.

Общий Makefile

Чтобы не повторять в каждом модуле логику «где ядро, какая архитектура, какой компилятор», в примерах есть общий `module.mk`:

docs/dev-book/examples/kmod/module.mk

```
# Shared wrapper for out-of-tree modules, built against a kernel built by the SDK.
#
#   make                  for the firmware selected in the SDK (output/.fw)
#   make FW=bbb           for another firmware
#   make install          into output/<fw>/linux-install/modules (goes into the image)
#   make SDK=/path/to/sdk  if this directory is not inside the SDK
#
# The module source list lives in the Kbuild file next to the Makefile.

SDK  ?= $(abspath $(CURDIR)/../../../../..)
```

```
FW   ?= $(shell cat $(SDK)/output/.fw 2>/dev/null)
KDIR ?= $(SDK)/output/$(FW)/linux

ifeq ($(wildcard $(KDIR)/.config),)
$(error Kernel for '$(FW)' not found in $(KDIR): run "make FW=$(FW) linux" in the SDK)
endif

# Architecture and compiler follow the kernel configuration
ifneq ($(shell grep -x 'CONFIG_ARM64=y' $(KDIR)/.config),)
ARCH          ?= arm64
CROSS_COMPILE ?= aarch64-linux-gnu-
else
ARCH          ?= arm
CROSS_COMPILE ?= arm-linux-gnueabihf-
endif

KMAKE = $(MAKE) -C $(KDIR) M=$(CURDIR) ARCH=$(ARCH) CROSS_COMPILE=$(CROSS_COMPILE)

all:
    $(KMAKE) modules

install: all
    $(KMAKE) INSTALL_MOD_PATH=$(SDK)/output/$(FW)/linux-install/modules INSTALL_MOD_STRIP=1 m
odules_install

clean:
    $(KMAKE) clean

.PHONY: all install clean
```

В каталоге каждого модуля — два крошечных файла:

```
Kbuild:    obj-m := hello.o
Makefile:  include ../module.mk
```

`Kbuild` читает система сборки ядра (какие объекты собрать), `Makefile` — вы (как и для какого ядра собрать). `make -C <каталог O= ядра> M=$(CURDIR)` — стандартный способ сборки внешнего модуля: каталог `output/<fw>/linux` содержит Makefile-обёртку, которая ведёт в исходники ядра.

Пример 1: hello

docs/dev-book/examples/kmod/hello/hello.c

```
// SPDX-License-Identifier: GPL-2.0
/* Minimal module with a parameter: insmod hello.ko who=OGA */
#define pr_fmt(fmt) KBUILD_MODNAME ": " fmt

#include <linux/init.h>
#include <linux/module.h>
#include <linux/moduleparam.h>
#include <linux/utsname.h>

static char *who = "SDK";
module_param(who, charp, 0444);     /* visible in /sys/module/hello/parameters/who */
MODULE_PARM_DESC(who, "Whom to greet");

static int __init hello_init(void)
{
    pr_info("hello, %s! kernel %s on %s\n", who, init_utsname()->release,
        init_utsname()->machine);
    return 0;
}

static void __exit hello_exit(void)
{
```

```
    pr_info("bye, %s\n", who);
}

module_init(hello_init);
module_exit(hello_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("SDK example: hello world module");
```

```
cd docs/dev-book/examples/kmod/hello
make FW=bbb                  # hello.ko для BBB
scp hello.ko root@board:/tmp/
```

```
# insmod /tmp/hello.ko who=BBB
# dmesg | tail -1
hello: hello, BBB! kernel 7.2.9-bbb on armv7l
# cat /sys/module/hello/parameters/who
BBB
# rmmod hello
```

- `pr_fmt` добавляет префикс `hello:` ко всем сообщениям модуля.
- `module_param(who, charp, 0444)` делает параметр, который задаётся при загрузке и виден в sysfs (только чтение).
- `MODULE_LICENSE("GPL")` обязателен: без него ядро «испортится» (taint) и модулю будут недоступны функции, экспортированные как `EXPORT_SYMBOL_GPL`.

Пример 2: символьное устройство

docs/dev-book/examples/kmod/sdkchar/sdkchar.c

```
// SPDX-License-Identifier: GPL-2.0
/*
 * /dev/sdkchar: a misc character device that keeps the last 4 KiB written to it.
 *
 *   echo hello > /dev/sdkchar ; cat /dev/sdkchar
 */
#define pr_fmt(fmt) KBUILD_MODNAME ": " fmt

#include <linux/fs.h>
#include <linux/miscdevice.h>
#include <linux/module.h>
#include <linux/mutex.h>

#define BUF_SIZE 4096

static char buf[BUF_SIZE];
static size_t buf_len;
static DEFINE_MUTEX(buf_lock);

static ssize_t sdkchar_read(struct file *f, char __user *ubuf, size_t len, loff_t *pos)
{
    ssize_t ret;

    mutex_lock(&buf_lock);
    ret = simple_read_from_buffer(ubuf, len, pos, buf, buf_len);
    mutex_unlock(&buf_lock);
    return ret;
}

static ssize_t sdkchar_write(struct file *f, const char __user *ubuf, size_t len, loff_t
*pos)
{
    ssize_t ret;
```

```
    mutex_lock(&buf_lock);
    if (*pos == 0)
        buf_len = 0;        /* a new write replaces the content */
    ret = simple_write_to_buffer(buf, BUF_SIZE, pos, ubuf, len);
    if (ret > 0)
        buf_len = max_t(size_t, buf_len, *pos);
    mutex_unlock(&buf_lock);
    return ret;
}

static const struct file_operations sdkchar_fops = {
    .owner  = THIS_MODULE,
    .read   = sdkchar_read,
    .write  = sdkchar_write,
    .llseek = default_llseek,
};

static struct miscdevice sdkchar_dev = {
    .minor  = MISC_DYNAMIC_MINOR,
    .name   = "sdkchar",        /* mdev creates /dev/sdkchar */
    .fops   = &sdkchar_fops,
    .mode   = 0666,
};

module_misc_device(sdkchar_dev);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("SDK example: misc character device");
```

```
# insmod sdkchar.ko
# ls -l /dev/sdkchar                          (младший номер у вас будет свой)
crw-rw-rw-    1 root     root       10, 123 ... /dev/sdkchar
# echo "hello from shell" > /dev/sdkchar
# cat /dev/sdkchar
hello from shell
```

`misc`-устройство — самый простой способ получить файл в `/dev`: ядро само выделит номер, а mdev создаст узел. `simple_read_from_buffer` и `simple_write_to_buffer` берут на себя проверку границ и копирование между пространствами пользователя и ядра (`copy_to_user`/`copy_from_user`). Мьютекс защищает буфер от одновременного доступа из нескольких процессов.

Пример 3: логгер кнопок (OGA и любая плата с клавиатурой)

docs/dev-book/examples/kmod/padlog/padlog.c

```
// SPDX-License-Identifier: GPL-2.0
/*
 * padlog: log every key/button press of every input device to the kernel log.
 *
 * An input *handler* sits next to evdev and receives the same events. On the
 * ODROID-GO Advance it shows the gamepad codes (BTN_DPAD_UP = 544, ...).
 */
#define pr_fmt(fmt) KBUILD_MODNAME ": " fmt

#include <linux/input.h>
#include <linux/module.h>
#include <linux/slab.h>

static void padlog_event(struct input_handle *handle, unsigned int type,
             unsigned int code, int value)
{
    if (type == EV_KEY && value == 1)
        pr_info("%s: code %u pressed\n", handle->dev->name, code);
}
```

```
static int padlog_connect(struct input_handler *handler, struct input_dev *dev,
              const struct input_device_id *id)
{
    struct input_handle *handle;
    int err;

    handle = kzalloc(sizeof(*handle), GFP_KERNEL);
    if (!handle)
        return -ENOMEM;
    handle->dev = dev;
    handle->handler = handler;
    handle->name = "padlog";

    err = input_register_handle(handle);
    if (err)
        goto err_free;
    err = input_open_device(handle);
    if (err)
        goto err_unregister;
    pr_info("watching %s\n", dev->name);
    return 0;

err_unregister:
    input_unregister_handle(handle);
err_free:
    kfree(handle);
    return err;
}

static void padlog_disconnect(struct input_handle *handle)
{
    input_close_device(handle);
    input_unregister_handle(handle);
    kfree(handle);
}

/* Any device that can send EV_KEY: buttons, keyboards */
static const struct input_device_id padlog_ids[] = {
    { .flags = INPUT_DEVICE_ID_MATCH_EVBIT, .evbit = { BIT_MASK(EV_KEY) } },
    { }
};
MODULE_DEVICE_TABLE(input, padlog_ids);

static struct input_handler padlog_handler = {
    .name       = "padlog",
    .event      = padlog_event,
    .connect    = padlog_connect,
    .disconnect = padlog_disconnect,
    .id_table   = padlog_ids,
};

static int __init padlog_init(void)
{
    return input_register_handler(&padlog_handler);
}

static void __exit padlog_exit(void)
{
    input_unregister_handler(&padlog_handler);
}

module_init(padlog_init);
module_exit(padlog_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("SDK example: log input key presses");
```

Input handler — «соседний» с evdev обработчик: ядро вызывает `connect()` для каждого подходящего устройства ввода и `event()` для каждого события. На OGA:

```
# insmod padlog.ko                          (примерный вывод)
padlog: watching gpio-keys
padlog: watching rk805 pwrkey
# (нажимаем крестовину вверх, затем одну из кнопок A/B/X/Y)
padlog: gpio-keys: code 544 pressed          BTN_DPAD_UP = 0x220
padlog: gpio-keys: code 305 pressed          BTN_EAST    = 0x131
```

Стика (`adc-joystick`) в списке не будет: он посылает только `EV_ABS`, а фильтр `padlog_ids` пропускает устройства, умеющие `EV_KEY`. Коды смотрите в `include/uapi/linux/input-event-codes.h`.

На Raspberry Pi 400 тот же модуль будет показывать коды встроенной клавиатуры, на BBB и RPi — любой подключённой USB-клавиатуры.

> **ЛОВУШКА: КОНТЕКСТ**
> `event()` вызывается в атомарном контексте (под спинлоком устройства). В нём нельзя спать: никаких `mutex_lock`, `kmalloc(GFP_KERNEL)`, `msleep`. Если нужно что-то тяжёлое — отложите в workqueue (как в примере 4).

Пример 4: драйвер по device tree

Настоящие драйверы не ищут железо сами: они говорят ядру, с какими узлами device tree работают (`compatible`), а ядро вызывает `probe()`, когда такой узел есть.

docs/dev-book/examples/kmod/sdk-gpio-led/sdk-gpio-led.c

```
// SPDX-License-Identifier: GPL-2.0
/*
 * sdk-gpio-led: platform driver that blinks a GPIO described in device tree.
 *
 *  sdk-led {
 *      compatible = "sdk,gpio-led";
 *      led-gpios = <&gpio 17 GPIO_ACTIVE_HIGH>;
 *  };
 *
 * MODULE_DEVICE_TABLE(of, ...) makes the module load automatically (mdev
 * runs modprobe for the node's modalias) once it is installed in the image.
 */
#include <linux/gpio/consumer.h>
#include <linux/jiffies.h>
#include <linux/mod_devicetable.h>
#include <linux/module.h>
#include <linux/platform_device.h>
#include <linux/workqueue.h>

static unsigned int period_ms = 1000;
module_param(period_ms, uint, 0644);    /* writable: /sys/module/sdk_gpio_led/parameters/
period_ms */
MODULE_PARM_DESC(period_ms, "Blink period in milliseconds");

struct sdk_led {
    struct gpio_desc *gpio;
    struct delayed_work work;
    bool on;
};

static void sdk_led_work(struct work_struct *work)
{
    struct sdk_led *led = container_of(to_delayed_work(work), struct sdk_led, work);

    led->on = !led->on;
    gpiod_set_value_cansleep(led->gpio, led->on);
    schedule_delayed_work(&led->work, msecs_to_jiffies(max(period_ms, 20U) / 2));
```

```
}

static int sdk_led_probe(struct platform_device *pdev)
{
    struct sdk_led *led;

    led = devm_kzalloc(&pdev->dev, sizeof(*led), GFP_KERNEL);
    if (!led)
        return -ENOMEM;

    /* "led-gpios" in DT; logical value 1 = on, GPIO_ACTIVE_LOW is handled for us */
    led->gpio = devm_gpiod_get(&pdev->dev, "led", GPIOD_OUT_LOW);
    if (IS_ERR(led->gpio))
        return dev_err_probe(&pdev->dev, PTR_ERR(led->gpio), "no led-gpios\n");

    INIT_DELAYED_WORK(&led->work, sdk_led_work);
    platform_set_drvdata(pdev, led);
    schedule_delayed_work(&led->work, 0);
    dev_info(&pdev->dev, "blinking, period %u ms\n", period_ms);
    return 0;
}

static void sdk_led_remove(struct platform_device *pdev)
{
    struct sdk_led *led = platform_get_drvdata(pdev);

    cancel_delayed_work_sync(&led->work);
    gpiod_set_value_cansleep(led->gpio, 0);
}

static const struct of_device_id sdk_led_of_match[] = {
    { .compatible = "sdk,gpio-led" },
    { }
};
MODULE_DEVICE_TABLE(of, sdk_led_of_match);

static struct platform_driver sdk_led_driver = {
    .probe  = sdk_led_probe,
    .remove = sdk_led_remove,
    .driver = {
        .name       = "sdk-gpio-led",
        .of_match_table = sdk_led_of_match,
    },
};
module_platform_driver(sdk_led_driver);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("SDK example: blink a GPIO from device tree");
```

Ключевые приёмы:

- `devm_*` — ресурсы, которые ядро освободит само при отвязке драйвера. Не нужно писать очистку для каждой ошибки в `probe()`.
- `devm_gpiod_get(dev, "led", ...)` берёт GPIO из свойства `led-gpios` узла. Полярность (`GPIO_ACTIVE_LOW`) учитывается автоматически.
- `dev_err_probe()` правильно обрабатывает `-EPROBE_DEFER` (контроллер GPIO ещё не готов — ядро попробует позже) и не засоряет лог.
- `delayed_work` — периодическая работа в контексте процесса, где можно спать (`gpiod_set_value_cansleep`).
- `MODULE_DEVICE_TABLE(of, ...)` записывает в модуль алиас `of:N*T*Csdk,gpio-led`. Когда в DT есть такой узел, mdev при загрузке вызовет `modprobe` с этим алиасом, и модуль загрузится сам — если он установлен в образ.

Чтобы драйвер заработал, в device tree платы должен появиться узел `sdk,gpio-led` — об этом глава 8.

Модули в образе

Пока модуль копируется руками, он живёт до перепрошивки. Чтобы модуль был в каждом образе и грузился сам:

```
make FW=rpi4 linux                                  # 1. ядро (стирает linux-install!)
make -C docs/dev-book/examples/kmod/sdk-gpio-led FW=rpi4 install   # 2. модуль
make FW=rpi4 image                                  # 3. образ
```

`make install` вызывает `modules_install` ядра: модуль попадает в `linux-install/modules/lib/modules/7.2.9-rpi4/updates/`, а `depmod` обновляет `modules.dep` и `modules.alias`. `mkimage.sh` копирует весь этот каталог в rootfs.

Рецепт: встроить свои модули в SDK

Ручной порядок из трёх команд легко нарушить. Его можно встроить в Makefile SDK — это не сделано в репозитории, но изменение небольшое. В конфиг прошивки:

```
LINUX_EXT_MODULES := apps/kmod/sdk-gpio-led apps/kmod/padlog
```

В Makefile SDK, в рецепт цели `linux`, после строки с `modules_install`:

```
    $(foreach m,$(LINUX_EXT_MODULES),\
      $(LINUX_MAKE) M=$(call sdkpath,$(m)) modules && \
      $(LINUX_MAKE) M=$(call sdkpath,$(m)) INSTALL_MOD_PATH=$(LINUX_INSTALL)/modules \
        INSTALL_MOD_STRIP=1 modules_install &&) true
```

`LINUX_MAKE` уже содержит `-C src/linux O=output/<fw>/linux ARCH=… CROSS_COMPILE=…`; вызов с `O=` и `M=` одновременно проверен — модуль собирается и ставится в `updates/`. Тогда `make linux image` всегда даёт образ, в котором ядро и модули согласованы.

Отладка модулей

- `dmesg -w` в отдельной консоли — смотреть сообщения по ходу.
- `cat /proc/modules`, `lsmod` — что загружено; `modinfo <модуль>` — параметры и алиасы.
- Oops (падение в ядре) печатает стек вызовов с адресами вида `sdkchar_write+0x24/0x90`. Строку исходника можно найти на ПК:

```
aarch64-linux-gnu-objdump -dS sdkchar.ko | less        # дизассемблер с исходником
src/linux/scripts/decode_stacktrace.sh output/oga/linux/vmlinux \
    src/linux docs/dev-book/examples/kmod/sdkchar < oops.txt
```

Отладочная информация (`CONFIG_DEBUG_INFO`) включена во всех ядрах SDK, поэтому `vmlinux` и модули содержат номера строк.

ГЛАВА 8

Правки ядра и device tree

Порядок работы с исходниками ядра

```
cd src/linux
git switch oga                          # все правки — в ветке oga
$EDITOR drivers/...                     # правка
cd ../..
make FW=rpi4 linux                      # инкрементальная сборка: секунды–минуты
make FW=rpi4 image                      # образ с новыми модулями
make FW=rpi4 flash-boot DEV=/dev/sdX    # или TFTP (глава 3)
```

Когда работает:

```
cd src/linux
git add -p && git commit -s -m "drivers: ..."
git format-patch -1                     # патч для отправки upstream или в другой проект
```

ЛОВУШКА: ОТНОСИТЕЛЬНЫЙ `O=`

Если запускаете make ядра руками, путь в `O=` должен быть **абсолютным**: `make -C src/linux O=output/oga/linux ...` трактует `O` относительно `src/linux` и создаст `src/linux/output/...` — и ещё пожалуется, что `.config` не найден. Цели SDK (`make linux` и др.) всегда передают абсолютные пути.

Опции ядра

`make linux-menuconfig` меняет `.config`, но при следующей смене фрагмента изменения пропадут (первая книга, глава 6). Перенесите их во фрагмент. Чтобы понять, что именно поменялось, есть `scripts/diffconfig`:

```
cp output/oga/linux/.config /tmp/before
make FW=oga linux-menuconfig
src/linux/scripts/diffconfig /tmp/before output/oga/linux/.config
#  USB_CONFIGFS m -> y
# +USB_CONFIGFS_F_HID y
```

Перепишите строки в формат фрагмента (`CONFIG_USB_CONFIGFS=y`) в `board/oga/linux/oga.config` и пересоберите: `kconfig-check.sh` сразу скажет, если какая-то опция не встала из-за зависимостей.

Свой драйвер в дереве ядра

Модуль из главы 7 можно перенести в дерево — тогда он собирается вместе с ядром, выбирается в menuconfig и может быть встроенным (`=y`).

```
cp docs/dev-book/examples/kmod/sdk-gpio-led/sdk-gpio-led.c src/linux/drivers/misc/
```

В `drivers/misc/Kconfig`:

```
config SDK_GPIO_LED
    tristate "OdroidLinux SDK example GPIO LED driver"
    depends on GPIOLIB && OF
    help
      Blinks a GPIO described by an "sdk,gpio-led" device tree node.
```

В `drivers/misc/Makefile`:

```
obj-$(CONFIG_SDK_GPIO_LED)  += sdk-gpio-led.o
```

И во фрагмент платы — `CONFIG_SDK_GPIO_LED=m` (или `=y`). Дальше `make linux image`.

Device tree: два способа

Устройство, которое ядро не может обнаружить само (светодиод на GPIO, датчик на I2C, дисплей на SPI), описывается в device tree. Есть два пути:

	Правка `.dts` платы	**Overlay (`.dtbo`)**
Где	`src/linux/arch/.../dts/...`, коммит в ветке `oga`	отдельный файл, накладывается при загрузке
Когда применяется	при сборке dtb	U-Boot перед стартом ядра (`fdtoverlays` в extlinux)
Плюсы	просто, видно в git ядра	не трогает ядро; можно включать и выключать на карте
Минусы	меняет dtb для всех	нужны `__symbols__` в базовом dtb

Правка `.dts`

Например, для Raspberry Pi 4 — в конец `arch/arm/boot/dts/broadcom/bcm2711-rpi-4-b.dts` (arm64-dts Pi 4 подключает этот файл из `arch/arm`):

```
/ {
    sdk-led {
        compatible = "sdk,gpio-led";
        led-gpios = <&gpio 17 GPIO_ACTIVE_HIGH>;
    };
};
```

`make linux image`, и после загрузки:

```
# ls /proc/device-tree/sdk-led/
compatible  led-gpios  name
# ls /sys/bus/platform/drivers/sdk-gpio-led/
bind  sdk-led  uevent  unbind              <- драйвер привязался к узлу
```

Overlay

Overlay — это фрагмент device tree с пометкой `/plugin/`, который ссылается на узлы базового dtb **по меткам** (`&gpio`, `&am33xx_pinmux`, `&i2c1`). Чтобы метки можно было найти, базовый dtb должен содержать таблицу `__symbols__`.

Плата	`__symbols__` в dtb SDK	Что сделать
BBB	есть	ничего
RPi 1, RPi 4/400	есть	ничего
OGA	**нет**	собрать ядро с `DTC_FLAGS=-@`

Проверка: `fdtget -l output/oga/linux-install/dtbs/rockchip/rk3326-odroid-go2-v11.dtb / | grep __symbols__`. Для OGA добавьте в `configs/oga_defconfig`:

```
LINUX_MAKE_ARGS := DTC_FLAGS=-@
```

Это проверено: с `DTC_FLAGS=-@` в dtb появляется `__symbols__`, без него — нет.

BeagleBone Black: светодиод на P9_12 с pinmux

docs/dev-book/examples/dts/bbb-sdk-led.dtso

```
// SPDX-License-Identifier: GPL-2.0
/*
 * BeagleBone Black: LED + resistor on P9_12 (GPIO1_28), driven by sdk-gpio-led.
 * P9_12 is pad gpmc_be1n; mode 7 makes it a GPIO.
 */
/dts-v1/;
/plugin/;

#include <dt-bindings/gpio/gpio.h>
#include <dt-bindings/pinctrl/am33xx.h>

&am33xx_pinmux {
    sdk_led_pins: sdk-led-pins {
        pinctrl-single,pins = <
            AM33XX_PADCONF(AM335X_PIN_GPMC_BEN1, PIN_OUTPUT, MUX_MODE7)
        >;
    };
};

&{/} {
    sdk-led {
        compatible = "sdk,gpio-led";
        pinctrl-names = "default";
        pinctrl-0 = <&sdk_led_pins>;
        led-gpios = <&gpio1 28 GPIO_ACTIVE_HIGH>;
    };
};
```

Обратите внимание на **pinmux**: на AM335x каждый вывод (pad) выполняет одну из восьми функций. `AM33XX_PADCONF(AM335X_PIN_GPMC_BEN1, PIN_OUTPUT, MUX_MODE7)` говорит контроллеру выводов: pad `gpmc_be1n` (это P9_12) — в режим 7 (GPIO), на выход. Драйвер `pinctrl-single` применит настройку, когда к узлу `sdk-led` привяжется драйвер (`pinctrl-names = "default"`).

Raspberry Pi 1 / 4 / 400: GPIO17

docs/dev-book/examples/dts/rpi-sdk-led.dtso

```
// SPDX-License-Identifier: GPL-2.0
/*
 * Raspberry Pi 1 / 4 / 400: LED + resistor on GPIO17 (header pin 11),
 * driven by sdk-gpio-led. The BCM283x/2711 pin controller switches the pin
 * to GPIO mode when the driver requests it, so no pinctrl node is needed.
 */
/dts-v1/;
```

```
/plugin/;

#include <dt-bindings/gpio/gpio.h>

&{/} {
    sdk-led {
        compatible = "sdk,gpio-led";
        led-gpios = <&gpio 17 GPIO_ACTIVE_HIGH>;
    };
};
```

ODROID-GO Advance: EEPROM на I2C разъёма P2

docs/dev-book/examples/dts/oga-i2c-eeprom.dtso

```
// SPDX-License-Identifier: GPL-2.0
/*
 * ODROID-GO Advance: 24C32 EEPROM on the expansion header P2
 * (pin 7 SCL, pin 8 SDA = i2c1, 3.3 V). Appears as
 * /sys/bus/i2c/devices/1-0050/eeprom.
 *
 * The OGA dtbs are built without __symbols__: build the kernel with
 * LINUX_MAKE_ARGS := DTC_FLAGS=-@ before applying overlays to them.
 */
/dts-v1/;
/plugin/;

&i2c1 {
    #address-cells = <1>;
    #size-cells = <0>;

    eeprom@50 {
        compatible = "atmel,24c32";
        reg = <0x50>;
        pagesize = <32>;
    };
};
```

Драйвер `at24` (модулем в ядре OGA) создаст файл `/sys/bus/i2c/devices/1-0050/eeprom`, который читается и пишется как обычный файл. Без overlay то же устройство можно «объявить» и вручную: `echo 24c32 0x50 > /sys/bus/i2c/devices/i2c-1/new_device`.

Сборка overlay

docs/dev-book/examples/dts/build-overlay.sh

```
#!/bin/sh
# Compile a .dtso overlay with the kernel's dt-bindings headers.
#   build-overlay.sh <linux source dir> <in.dtso> <out.dtbo>
set -e
LINUX=$1 IN=$2 OUT=$3
cpp -nostdinc -undef -D__DTS__ -x assembler-with-cpp \
    -I "$LINUX/include" -I "$LINUX/scripts/dtc/include-prefixes" "$IN" |
    dtc -@ -q -I dts -O dtb -o "$OUT" -
```

```
docs/dev-book/examples/dts/build-overlay.sh src/linux \
    docs/dev-book/examples/dts/bbb-sdk-led.dtso bbb-sdk-led.dtbo
```

Препроцессор C нужен, чтобы раскрыть `#include` и макросы (`GPIO_ACTIVE_HIGH`, `AM33XX_PADCONF`), а `dtc -@` сохраняет ссылки на метки для применения позже.

Проверка на ПК

Ещё до платы overlay можно наложить на настоящий dtb утилитой `fdtoverlay`:

```
fdtoverlay -i output/bbb/linux-install/dtbs/ti/omap/am335x-boneblack.dtb \
           -o /tmp/merged.dtb bbb-sdk-led.dtbo
fdtget /tmp/merged.dtb /sdk-led compatible          # sdk,gpio-led
```

Так проверены все три overlay из этой главы: BBB-overlay — на `am335x-boneblack.dtb`, RPi-overlay — на dtb RPi 1 B+, B rev2, 4 B и 400, OGA-overlay — на dtb Black Edition, собранный с `-@`. Если наложить на OGA обычный dtb SDK, `fdtoverlay` ответит `FDT_ERR_BADOFFSET` — меток нет.

Применение при загрузке

U-Boot применяет overlay, перечисленные в строке `fdtoverlays` метки extlinux. `mkimage.sh` генерирует `extlinux.conf` без неё, поэтому:

1. Положите `.dtbo` в BOOT, например через конфиг прошивки: `IMAGE_BOOT_FILES += $(SDK_DIR)/bbb-sdk-led.dtbo` (или скопируйте на карту руками).
2. Добавьте строку в `extlinux/extlinux.conf` на карте:

```
label linux
    kernel /zImage
    fdtdir /
    fdtoverlays /bbb-sdk-led.dtbo
    append root=PARTUUID=42424231-02 ...
```

U-Boot загружает overlay по адресу из переменной `fdtoverlay_addr_r`:

Плата	`fdtoverlay_addr_r` в окружении U-Boot
OGA	есть (`0x01f00000`)
BBB	есть (`0x89000000`)
RPi 1, RPi 4	**нет** — U-Boot напишет `Invalid fdtoverlay_addr_r` и пропустит overlay

Для Raspberry Pi задайте адрес сами, например сразу после области dtb (`fdt_addr_r=0x05600000`, под неё отведён 1 МиБ):

```
U-Boot> setenv fdtoverlay_addr_r 0x05700000
U-Boot> saveenv
```

`0x05700000` совпадает с `ramdisk_addr_r`, но initrd SDK не использует.

> **СДЕЛАТЬ ЭТО ЧАСТЬЮ SDK**
> Хороший следующий шаг для SDK — переменная `IMAGE_FDTOVERLAYS` в конфиге прошивки, по которой `mkimage.sh` кладёт `.dtbo` в BOOT и дописывает строку `fdtoverlays` в `extlinux.conf`. Изменение — десяток строк в `mkimage.sh`.

Отладка ядра

Что	Как
Лог с самого начала, даже до драйвера UART	добавить `earlycon` в `LINUX_CMDLINE`: ядро возьмёт порт и тип из `stdout-path` (работает на всех четырёх платах)
Все сообщения на консоль	`ignore_loglevel` в командной строке или `dmesg -n 8`
Почему не привязался драйвер	`cat /sys/kernel/debug/devices_deferred` (после `mount -t debugfs none /sys/kernel/debug`)
Сообщения `pr_debug`/`dev_dbg`	`CONFIG_DYNAMIC_DEBUG=y` (включён только в ядре RPi 1, остальным — во фрагмент), затем `echo 'module sdk_gpio_led +p' > /sys/kernel/debug/dynamic_debug/control`
Какие опции у работающего ядра	`zcat /proc/config.gz` (OGA, BBB, RPi 4; в ядре RPi 1 нет `IKCONFIG_PROC`)
Расшифровать oops	`scripts/decode_stacktrace.sh output/<fw>/linux/vmlinux src/linux < oops.txt`

EARLYCON

Ранняя консоль нужна, когда ядро падает до того, как поднялся настоящий драйвер UART: без неё на экране после `Starting kernel ...` — тишина. Для всех плат SDK достаточно слова `earlycon`: в их DTS есть `stdout-path`, а драйверы UART (`8250` для Rockchip и AM335x, `pl011` для RPi 1, `bcm2835aux` для RPi 4) объявляют раннюю консоль.

ГЛАВА 9

Правки в U-Boot

Загрузчик менять страшнее всего: сломанный U-Boot — и плата не дойдёт до Linux. Поэтому сначала о том, как безопасно проверять изменения на каждой плате.

Какой U-Boot на самом деле работает

Плата	Откуда берётся U-Boot	Что пересобрать и записать
OGA, заводской SPI	`uboot.img` с сектора 16384 (U-Boot proper, без SPL)	`make uboot image` + `flash-boot`
OGA, пустой SPI	idbloader (TPL+SPL) с сектора 64 + FIT с сектора 2048	то же
BBB	файлы `MLO` и `u-boot.img` на разделе BOOT	`make uboot`, скопировать два файла на FAT
RPi 1, RPi 4/400	файл `u-boot.bin` на разделе BOOT (его запускает прошивка GPU)	`make uboot`, скопировать файл на FAT

OGA: правки в SPL и TPL не видны с заводским SPI

На Black Edition с заводским загрузчиком во SPI DDR инициализирует блоб Rockchip, а SPL не запускается вовсе — работает только U-Boot proper из `uboot.img`. Правка в `arch/arm/mach-rockchip/` для SPL или в драйвере DDR на такой консоли ничего не изменит. Проверить, что вообще работает SPL, можно по логу: строки `U-Boot SPL 2026.07` при загрузке нет.

Как восстановиться, если U-Boot сломан

Плата	Восстановление
OGA	SD-карта съёмная, SPI мы не трогаем: `make flash-boot` старого образа через картридер
BBB	без кнопки S2 плата загрузится с eMMC; карту исправить на ПК (вернуть `MLO`, `u-boot.img`)
RPi	вернуть `u-boot.bin` на карте; в крайнем случае `config.txt`: `kernel=zImage` / `Image` — прошивка загрузит ядро напрямую

Держите запасную карту

Самая дешёвая страховка при работе с загрузчиком — вторая SD-карта с заведомо рабочим образом.

Окружение

U-Boot хранит переменные (`bootcmd`, `bootdelay`, адреса, ваши настройки) в «окружении»:

Плата	Где хранится	Когда сбрасывается
OGA	сырая область MMC, 16–32 КиБ от начала карты	`make flash` (образ пишет туда нули)
BBB, RPi	файл `uboot.env` на разделе BOOT	`make flash`, `make flash-boot` (перезаписывают BOOT)

```
=> printenv                         всё окружение
=> printenv fdtfile board_name
=> setenv bootdelay 5
=> saveenv                          записать
=> env default -a; saveenv          вернуть значения по умолчанию
```

Пока `saveenv` не вызывали, U-Boot при старте пишет `Loading Environment from ... *** Warning - bad CRC, using default environment`. Это нормально.

Значения по умолчанию лучше задавать не в сохранённом окружении, а в конфиге — тогда они переживут перепрошивку. Например, во фрагмент `board/oga/u-boot/oga.config`:

```
CONFIG_BOOTDELAY=5
```

Полезные команды

```
=> bootflow scan -l           что U-Boot нашёл для загрузки (SD, сеть...)
=> bootflow scan -lb          найти и загрузить первое
=> mmc list ; mmc dev 0 ; mmc info
=> ls mmc 0:1 /               файлы на разделе BOOT
=> load mmc 0:1 ${kernel_addr_r} /Image
=> md.l ${fdt_addr_r} 10      дамп памяти
=> fdt addr ${fdt_addr_r}; fdt print /chosen
=> gpio status -a             линии GPIO
=> i2c bus ; i2c dev 1 ; i2c probe
=> dm tree                    устройства в модели драйверов U-Boot
=> bdinfo                     память, адреса
```

На OGA есть ещё `led list` и `led <имя> on|off` (`CMD_LED`) и `sf probe` для SPI-флеша.

Пример: своя команда

Команда `sdkinfo` печатает то, что U-Boot выяснил о плате: имя, выбранный для ядра dtb и объём памяти. Это хороший шаблон для любой своей диагностики.

docs/dev-book/examples/uboot/cmd/sdkinfo.c

```c
// SPDX-License-Identifier: GPL-2.0+
/*
 * sdkinfo: print what U-Boot knows about the board before booting Linux.
 * Goes to cmd/sdkinfo.c; see cmd/Kconfig and cmd/Makefile snippets.
 */
#include <command.h>
#include <env.h>
#include <vsprintf.h>
#include <asm/global_data.h>

DECLARE_GLOBAL_DATA_PTR;

static const char *env_or(const char *name)
{
    const char *v = env_get(name);
```

```
    return v ? v : "(not set)";
}

static int do_sdkinfo(struct cmd_tbl *cmdtp, int flag, int argc, char *const argv[])
{
    printf("board:     %s\n", env_or("board_name"));
    printf("fdtfile:   %s\n", env_or("fdtfile"));
    printf("RAM:       %lu MiB\n", (unsigned long)(gd->ram_size >> 20));
    printf("bootdelay: %s\n", env_or("bootdelay"));
    if (argc > 1)
        printf("note:      %s\n", argv[1]);
    return CMD_RET_SUCCESS;
}

U_BOOT_CMD(sdkinfo, 2, 0, do_sdkinfo,
       "show board info collected by U-Boot",
       "[note]\n"
       "    - print board name, selected dtb, RAM size; echo an optional note");
```

`U_BOOT_CMD(имя, макс. аргументов, повторять по Enter, функция, кратко, подробно)` регистрирует команду. `gd` — глобальные данные U-Boot, в том числе размер RAM.

Подключение — в `src/u-boot` (ветка `oga`):

```
cp docs/dev-book/examples/uboot/cmd/sdkinfo.c src/u-boot/cmd/
```

В `cmd/Kconfig` (например, перед `config CMD_SLEEP`):

docs/dev-book/examples/uboot/cmd/Kconfig.snippet

```
config CMD_SDKINFO
    bool "sdkinfo"
    default n
    help
      Print the board name, the dtb U-Boot selected for Linux and the
      RAM size (OdroidLinux SDK example).
```

В `cmd/Makefile`:

docs/dev-book/examples/uboot/cmd/Makefile.snippet

```
obj-$(CONFIG_CMD_SDKINFO) += sdkinfo.o
```

Включить для платы — строка во фрагменте U-Boot (у OGA он уже есть, для других плат создайте `board/<плата>/u-boot/<плата>.config` и укажите его в `UBOOT_FRAGMENTS`):

```
CONFIG_CMD_SDKINFO=y
```

```
make FW=oga uboot image && make FW=oga flash-boot DEV=/dev/sdX
```

Вывод (примерный; имя платы и dtb у каждой свои):

```
=> help sdkinfo
sdkinfo - show board info collected by U-Boot
=> sdkinfo
board:     <имя из таблицы go2.c>
fdtfile:   rockchip/rk3326-odroid-go2-v11.dtb
RAM:       1022 MiB
bootdelay: 2
```

На BBB `board_name` берётся из EEPROM платы (`A335BNLT` у BeagleBone Black), на Raspberry Pi — из таблицы ревизий в `board/raspberrypi/rpi/rpi.c`.

> **ЧТО ПРОВЕРЕНО**
> Команда собрана для OGA (`odroid-go2_defconfig` + фрагмент SDK, aarch64) и для RPi 1 (`rpi_defconfig`, arm) без предупреждений; строка справки есть в `u-boot.bin`. На железе не запускалась.

Пример: меню с запасным ядром

Когда экспериментируете с ядром, полезно держать на карте **заведомо рабочее** ядро и выбирать при загрузке. U-Boot понимает меню в `extlinux.conf`:

```
menu title OdroidLinux boot
timeout 50
default stable

label stable
    menu label Stable kernel
    kernel /Image
    fdtdir /
    append root=PARTUUID=4f474131-02 rootwait rw console=ttyS2,115200n8 console=tty1
fbcon=rotate:3

label test
    menu label Test kernel
    kernel /Image-test
    fdtdir /
    append root=PARTUUID=4f474131-02 rootwait rw console=ttyS2,115200n8 earlycon
ignore_loglevel
```

`timeout` задаётся в десятых долях секунды. Меню показывается на консоли UART: пункты пронумерованы, выбор — вводом номера. Новое ядро копируйте на BOOT как `Image-test` — и если оно не загрузится, следующая загрузка по умолчанию пойдёт со стабильным.

> **МОДУЛИ**
> Оба ядра используют одни и те же `/lib/modules` в rootfs. Если версия тестового ядра другая (например, сменили `LOCALVERSION`), его модули нужно доложить в rootfs отдельно.

Отладка самого U-Boot

- `CONFIG_LOG=y` и команда `log level 7` — подробный лог подсистем.
- `#define DEBUG` в начале `.c`-файла включает `debug()` в этом файле.
- Если U-Boot падает раньше консоли, помогает **debug UART** (`CONFIG_DEBUG_UART=y`): он работает с первой инструкции. У OGA он уже включён (первая книга, глава 7).
- `bdinfo` и `dm tree` показывают, что U-Boot думает о памяти и устройствах.

ГЛАВА 10

Особенности плат: сводка для разработчика

Сравнение

	OGA	BBB	RPi 1	RPi 4 / 400
CPU	4 × Cortex-A35, arm64	1 × Cortex-A8, arm	1 × ARM1176, armv6	4 × Cortex-A72, arm64
Тулчейн программ	`aarch64-buildroot-linux-gnu-`	`arm-buildroot-linux-gnueabihf-` (A8)	`arm-buildroot-linux-gnueabihf-` (ARMv6)	`aarch64-buildroot-linux-gnu-`
Ядро / vermagic	`Image`, SMP preempt	`zImage`, SMP, modversions, ARMv6	`zImage`, ARMv6, без SMP	`Image`, SMP preempt
Консоль	`ttyS2`	`ttyS0` (J1)	`ttyAMA0`	`ttyS1` (mini-UART)
Сеть	нет (USB-Ethernet)	Ethernet	Ethernet на USB (B/B+)	Ethernet
Загрузка по сети	нет	да	да (`usb start`)	да
Окружение U-Boot	MMC, 16 КиБ	`uboot.env` на FAT	`uboot.env` на FAT	`uboot.env` на FAT
`__symbols__` в dtb	нет (`DTC_FLAGS=-@`)	есть	есть	есть
`fdtoverlay_addr_r`	есть	есть	нет	нет
`/proc/config.gz`	есть	есть	нет	есть
Dynamic debug	нет	нет	есть	нет

ODROID-GO Advance

- **Загрузчик.** С заводским SPI работает только U-Boot proper из `uboot.img`; TPL/SPL не запускаются (глава 9). Правки в SPL проверяйте, только если стёрли SPI.
- **Экран** повёрнут: framebuffer в родной (портретной) ориентации панели, консоль повёрнута `fbcon=rotate:3` (нужен `CONFIG_FRAMEBUFFER_CONSOLE_ROTATION`, он включён во фрагменте OGA). Учитывайте это в графике (пример `oga-pad`).
- **Ввод**: `gpio-keys` (D-pad, A/B/X/Y, F1–F6, курки), `adc-joystick` (стик, `ABS_X/Y`), `rk805 pwrkey` (кнопка питания).
- **Разъём P2**: UART1 (`ttyS1`, контакты 2 — RXD, 3 — TXD; 4, 5 — CTS, RTS) и I2C1 (контакты 7 — SCL, 8 — SDA, 400 кГц). Свободных GPIO нет.
- **Звук** — кодек PMIC RK817, модулем; **зарядка** — `CHARGER_RK817`, модулем (`/sys/class/power_supply/`).

- **Светодиоды** — `blue:status` (PWM) и `red:charging`.
- **Сети нет**: Wi-Fi ESP8089 Black Edition в mainline не поддерживается. Для разработки — USB-Ethernet адаптер (драйверы RTL8152, AX88179, CDC модулями), UART (`lrzsz`) или картридер.
- **Overlay**: в dtb SDK нет `__symbols__`, нужен `LINUX_MAKE_ARGS := DTC_FLAGS=-@`.
- **USB OTG**: контроллер DWC2 включён в двухрежимной конфигурации, configfs-гаджеты (ECM, NCM, ACM) собраны модулями. Работает ли гаджет через разъём OGA, не проверено.

BeagleBone Black

- **Порядок загрузки**: eMMC раньше SD. Для загрузки с SD держите S2 при включении питания (или сотрите загрузчик на eMMC).
- **U-Boot** — файлы `MLO` и `u-boot.img` на FAT. Обновлять можно просто копированием.
- **Pinmux**: почти все выводы P8/P9 многофункциональные. Чтобы вывод стал GPIO, I2C, UART или PWM, нужен узел pinctrl в device tree (пример — overlay для P9_12).
- **GPIO**: четыре банка по 32 линии (`GPIO1_28` = банк 1, линия 28). Номер `/dev/gpiochipN` не обязан совпадать с номером банка — сверяйтесь с `gpiodetect`.
- **Ядро** из `omap2plus_defconfig`: базовая архитектура ARMv6 (ради старых OMAP2) и `MODVERSIONS`. Модули — строго от того же ядра.
- **HDMI** — tilcdc + TDA998x, модулями.
- **USB**: порт USB0 (mini-USB) — OTG на контроллере MUSB (модулем), configfs-гаджеты собраны: можно сделать сеть по USB-кабелю.
- **PRU** (два 200-МГц микроконтроллера реального времени) в этой книге не рассматриваются: для них нужен отдельный тулчейн и `remoteproc`.

Raspberry Pi 1 / Zero

- **ARMv6, одно ядро.** Тулчейн RPi 1 не взаимозаменяем с тулчейном BBB, хотя префикс одинаковый.
- **Загрузка**: прошивка GPU → `u-boot.bin` → extlinux. Прошивке нужен dtb со «старым» именем `bcm2708-rpi-*.dtb`; SDK кладёт копии mainline dtb под этими именами.
- **Ethernet** (только B/B+) висит на USB: в U-Boot сначала `usb start`, в ядре драйвер `USB_NET_SMSC95XX` встроен (годится для NFS-корня).
- **Overlay** U-Boot применяет (нужен `fdtoverlay_addr_r`); в самом ядре `CONFIG_OF_OVERLAY` выключен — во время работы overlay не загрузить.
- **Отладка**: `DYNAMIC_DEBUG` включён, `/proc/config.gz` — нет.
- **Zero / Zero W**: USB OTG (DWC2) — можно сделать USB-гаджет (`USB_ETH` модулем).
- **GPIO17** — контакт 11 гребёнки; GPIO-контроллер — `/dev/gpiochip0`.

Raspberry Pi 4 / 400

- **Загрузка**: EEPROM → `start4.elf` (читает `config.txt`) → `u-boot.bin` (64 бита, `arm_64bit=1`) → extlinux. Прошивке нужен `bcm2711-rpi-*.dtb` в корне BOOT, U-Boot — в `broadcom/` (`IMAGE_DTB_LAYOUT := both`).
- **Консоль на GPIO14/15** — mini-UART (`ttyS1`): его частота зависит от частоты ядра VideoCore, поэтому в `config.txt` стоит `enable_uart=1`.
- **Pi 400**: светодиода ACT нет (удалён в DTS), только PWR. Клавиатура — USB через контроллер VL805 на PCIe; PCIe, xHCI и HID встроены в ядро фрагментом SDK, чтобы клавиатура работала на `tty1` сразу.
- **Память U-Boot**: ядро грузится с `0x80000`, следующая занятая область — `scriptaddr=0x05400000`, то есть под `Image` около 83 МиБ (сейчас он ≈ 41 МиБ).
- **Overlay**: задайте `fdtoverlay_addr_r` (глава 8).
- **NFS-корень**: GENET и PHY встроены фрагментом SDK — можно грузиться по сети.
- **HDMI-звук** — через VC4 (встроен).

ГЛАВА 11

Практикум

Где держать свой код

Предлагаемая раскладка в репозитории SDK:

```
apps/                              исходники своих программ (по каталогу на программу)
apps/kmod/                         свои модули ядра вне дерева (+ module.mk)
board/common/buildroot/package/    пакеты для своих программ
board/common/buildroot/rootfs-overlay/
board/<плата>/...                  конфиги и фрагменты плат
src/linux, src/u-boot              правки ядра и загрузчика — в ветке oga, push в свои форки
```

Коммиты в SDK и в сабмодули — отдельные. Порядок, когда менялось ядро:

```
cd src/linux && git commit ... && git push mine oga && cd ../..
git add src/linux apps board configs && git commit -m "..." && git push
```

Чек-лист перед коммитом

Изменение	Проверить
Программа	собрана тулчейном прошивки; `file` показывает нужную архитектуру; работает на плате
Пакет	`make rootfs-make BR=<пакет>-rebuild`; файл есть в `rootfs.tar` (`tar -tvf`)
Модуль	собран против текущего ядра (`modinfo -F vermagic`); `insmod`/`rmmod` без oops
Фрагмент ядра/U-Boot	нет `WARNING` от `kconfig-check.sh`
Device tree	`fdtget` на собранном dtb показывает узел; драйвер привязался (`/sys/bus/.../drivers/`)
U-Boot	загрузка доходит до `login:`; запасная карта под рукой
Образ	`make image` без ошибок; при больших изменениях — полная перепрошивка `make flash`

Задания

1. ВСЕ ПЛАТЫ: HELLO ВЕЗДЕ

Соберите `hello` для каждой из четырёх прошивок и проверьте `file`. Запустите BBB-версию на RPi 1 (если есть обе) и объясните ошибку.

2. ВСЕ ПЛАТЫ: ПАКЕТ И INIT-СКРИПТ

Перенесите `sdk-hello` в `board/common/buildroot/package`, `hello` — в `apps/hello`, включите пакет в defconfig одной платы. Убедитесь, что при загрузке в `/var/log/messages` появляется строка от `sdk-hello`.

3. OGA: ИГРА

На основе `oga-pad` сделайте «змейку»: таймер через `poll` с тайм-аутом, движение по крестовине, перезапуск по Start. Добавьте звук: `aplay` короткого WAV через `system()` или ALSA API.

4. OGA: ДАТЧИК НА I2C

Подключите к P2 (контакты 7, 8, питание 3,3 В) любой I2C-датчик, найдите его `i2cdetect -y 1` (пакет i2c-tools), затем опишите его overlay и найдите драйвер в ядре по `compatible`.

5. BBB: СВЕТОДИОД ПО DEVICE TREE

Соберите `sdk-gpio-led` в образ (`make install + make image`), подключите светодиод к P9_12, примените overlay через `fdtoverlays`. Поменяйте частоту на лету: `echo 200 > /sys/module/sdk_gpio_led/parameters/period_ms`.

6. BBB ИЛИ RPI 4: NFS-КОРЕНЬ

Настройте TFTP + NFS (глава 3) и сделайте цикл «правка драйвера → перезагрузка» без извлечения карты.

7. RPI 1 / 4: КНОПКА

Подключите кнопку к GPIO27 и GND. В device tree опишите её как `gpio-keys` с `linux,code = <KEY_ENTER>` и `bias-pull-up` в pinctrl (или внешним резистором). Проверьте `evtest` и модулем `padlog`.

8. RPI 400: СВОЯ КОМАНДА U-BOOT

Добавьте `sdkinfo` во фрагмент U-Boot RPi 4. Задайте `fdtoverlay_addr_r` по умолчанию: допишите строку `fdtoverlay_addr_r=0x05700000` в файл окружения платы `src/u-boot/board/raspberrypi/rpi/rpi.env` (коммит в ветке `oga`). Проверьте `sdkinfo` и `printenv fdtoverlay_addr_r` после `env default -a`.

9. SDK: IMAGE_FDTOVERLAYS

Реализуйте в `mkimage.sh` переменную `IMAGE_FDTOVERLAYS`: собранные `.dtbo` — в BOOT, строка `fdtoverlays` — в `extlinux.conf`. Проверьте на BBB.

10. SDK: LINUX_EXT_MODULES

Встройте сборку своих модулей в цель `linux` по рецепту главы 7 и убедитесь, что после `make linux image` модуль `sdk-gpio-led` загружается сам.

ПРИЛОЖЕНИЕ А

Шпаргалка разработчика

Программы

```
export PATH=$SDK/output/<fw>/buildroot/host/bin:$PATH
<префикс>-gcc -O2 -g -o app app.c $(pkg-config --cflags --libs <lib>)
cmake -S . -B build -DCMAKE_TOOLCHAIN_FILE=$SDK/output/<fw>/buildroot/host/share/buildroot/
toolchainfile.cmake
scp app root@board:/usr/bin/
gdbserver :2345 app            # плата
<префикс>-gdb app              # ПК: set sysroot …/staging ; target remote board:2345
```

Пакеты

```
make rootfs-menuconfig && make rootfs-savedefconfig
make rootfs-make BR=<пакет>-rebuild        # после правки исходников
make rootfs-make BR=<пакет>-reconfigure    # после смены опций пакета
make rootfs image
```

Модули

```
cd apps/kmod/<модуль> && make FW=<fw>          # .ko против output/<fw>/linux
make FW=<fw> install                            # в linux-install (после make linux!)
insmod x.ko param=val ; rmmod x ; modinfo x.ko ; dmesg -w
```

Ядро и device tree

```
make linux-menuconfig ; src/linux/scripts/diffconfig before after
make linux image && make flash-boot DEV=/dev/sdX
build-overlay.sh src/linux x.dtso x.dtbo
fdtoverlay -i base.dtb -o merged.dtb x.dtbo && fdtget merged.dtb /node prop
fdtget -l <dtb> / | grep __symbols__
```

Командная строка ядра для отладки: `earlycon ignore_loglevel initcall_debug`.

U-Boot

```
=> printenv / setenv / saveenv / env default -a
=> bootflow scan -l ; bootflow scan -lb
=> setenv autoload no; dhcp; setenv serverip <ПК>; pxe get; pxe boot
=> tftpboot ${kernel_addr_r} <файл> ; booti|bootz ${kernel_addr_r} - ${fdt_addr_r}
=> ls mmc 0:1 / ; load mmc 0:1 <addr> <файл> ; md.l <addr> 10
=> gpio status -a ; i2c dev 1 ; i2c probe ; dm tree ; bdinfo
```

Доставка

Что	Команда
Весь образ	`make flash DEV=/dev/sdX`
Загрузчик + ядро + dtb	`make flash-boot DEV=/dev/sdX`
Файл по сети	`scp file root@board:/path`
Файл по UART	`picocom --send-cmd "sz -vv"` + `rz` на плате
Ядро по сети	TFTP + `pxe boot`
Весь rootfs по сети	NFS, `root=/dev/nfs nfsroot=… ip=dhcp`

ПРИЛОЖЕНИЕ В

Указатель примеров

Все файлы лежат в `docs/dev-book/examples/`.

Файл	Что это	Платы	Глава
`app/hello/`	минимальная программа + Makefile с `?=` и `DESTDIR`	все	4, 6
`app/cmake-demo/`	CMake-проект для toolchain-файла Buildroot	все	4
`app/gpio-blink/gpio-blink.c`	мигание GPIO через `/dev/gpiochipN` (uAPI v2)	BBB, RPi	5
`app/oga-pad/oga-pad.c`	evdev + framebuffer + поворот экрана	OGA	5
`app/serial-echo/serial-echo.c`	эхо по UART (termios, raw)	OGA (`ttyS1`), все	5
`package/sdk-hello/`	пакет Buildroot: `generic-package`, `SITE_METHOD=local`, init-скрипт	все	6
`kmod/module.mk`	сборка модулей против ядра SDK	все	7
`kmod/hello/`	модуль с параметром	все	7
`kmod/sdkchar/`	misc-устройство `/dev/sdkchar`	все	7
`kmod/padlog/`	input handler: лог нажатий	OGA, все с клавиатурой	7
`kmod/sdk-gpio-led/`	платформенный драйвер по device tree, `devm_*`, workqueue	BBB, RPi	7, 8
`dts/bbb-sdk-led.dtso`	overlay: P9_12 с pinmux	BBB	8
`dts/rpi-sdk-led.dtso`	overlay: GPIO17	RPi 1, 4, 400	8
`dts/oga-i2c-eeprom.dtso`	overlay: EEPROM на I2C1 разъёма P2	OGA	8
`dts/build-overlay.sh`	сборка `.dtso` → `.dtbo` с заголовками ядра	все	8
`uboot/cmd/sdkinfo.c`	своя команда U-Boot + фрагменты Kconfig/Makefile	все	9

Что и как проверено

Проверка	Результат
Программы `app/*` тулчейнами Buildroot `oga`, `bbb`, `rpi1`, `rpi4` (`-Wall -Wextra`)	собраны, предупреждений нет
`cmake-demo` с toolchain-файлом каждой прошивки	собран
Модули `kmod/*` против ядер `oga`, `bbb`, `rpi1`, `rpi4`	16 сборок, предупреждений нет
Установка модуля через `modules_install` с `O=` и `M=`	ставится в `updates/`, `depmod` отрабатывает
Overlay: `fdtoverlay` на dtb BBB, RPi 1 B+/B rev2, RPi 4 B, 400	узел `sdk-led` на месте
Overlay OGA: на dtb, собранный с `-@`	узел `eeprom@50` на месте; на dtb SDK — `FDT_ERR_BADOFFSET`
`DTC_FLAGS=-@` при сборке dtb ядра OGA	`__symbols__` появляется
Команда `sdkinfo` в U-Boot для OGA и RPi 1	собрана, предупреждений нет
Пакет `sdk-hello` в отдельной сборке Buildroot с тулчейном OGA	собран и установлен; `-rebuild` подхватывает правки, обычный `make` — нет
Запуск на железе	**не выполнялся**